АРХИТЕКТУРНЫЙ УЧЕБНИК И СПРАВОЧНИК

DX Framework

Как устроен движок: код, математика, рендер, физика, ECS и инженерные решения

DX_FrameWork

Издание 3.0 · срез 18 сентября 2026 года

Содержание

- Часть I. Паспорт движка
- 1. Что это за проект
- 2. Как пользоваться книгой после долгого перерыва
- 3. Как движок вырос: история по локальному git
- 4. Карта репозитория
- 5. Владение и жизненный цикл приложения
- 6. Сборка и запуск
- 7. Один кадр: фактический timeline
- 8. Время и численная устойчивость
- Часть II. C++ и ECS именно в этом проекте
- 9. C++: значения, ссылки, указатели и RAII
- 10. Как устроена ECS
- 11. Как добавить компонент и систему
- Часть III. Математический фундамент
- 12. Единицы и координатные соглашения
- 13. Векторы
- 14. Матрицы и Transform
- 15. Euler angles и quaternion
- 16. Камера и проекция
- 17. Bounds и frustum culling
- Часть IV. DirectX 11 и рендер
- 18. Ресурсы и состояния DirectX 11
- 19. Фактический render pipeline
- 20. PBR: математика Cook–Torrance
- 21. Цвет, HDR и IBL
- 22. Тени
- 23. Прозрачность, reflection и refraction
- 24. Частицы и взрывы
- 25. Скелетная анимация
- Часть V. Bullet и симуляция мира
- 26. Bullet Physics и синхронизация
- 27. Character controller
- 28. Давление, затопление и плавучесть
- 29. Комнаты и атмосфера
- 30. Энергетика
- 31. Damage, cutting и repair
- Часть VI. NPC, данные и инструменты

- 32. NPC AI pipeline
- 33. StateSystem и исполнение
- 34. Recast/Detour NavMesh
- 35. JobSystem и асинхронные конвейеры
- 36. Scene save, runtime save и Entity remap
- 37. Editor, picking и диагностика
- Часть VII. Срез качества и план развития
- 38. Сильные стороны движка
- 39. Реестр слабых мест
- 40. Производительность без угадывания
- 41. Как диагностировать симптом
- 42. Практикум: безопасное добавление engine-функции
- 43. Карта чтения кода
- Часть VIII. Работа с кодом и практикум по NPC
- 44. Как изучать собственный движок, а не просто читать файлы
- 45. Анатомия персонажа в DX Framework
- 46. Дружественный NPC в текущей архитектуре
- 47. Проектирование врага на существующем NPC pipeline
- 48. Практическое задание Первый полноценный дружественный NPC
- 49. Практическое задание Первый преследующий и атакующий враг
- 50. План одного рабочего дня и порядок сдачи
- Приложение A. Горячие клавиши
- Приложение B. Формулы на одной странице
- Приложение C. Словарь
- Приложение D. Что открыть через год

Часть I. Паспорт движка

1. Что это за проект

DX Framework — учебный игровой движок и одновременно основа космического симулятора. Он написан на C++20 для Windows и использует DirectX 11, DirectXMath, Bullet Physics, Assimp, TinyGLTF, Recast/Detour, ImGui, DirectWrite и JSON. Это уже не набор отдельных уроков: рендер, ECS, физика, редактор, импорт корабля, персонажи, NPC, атмосфера, энергетика, повреждения и сохранение сцены соединены одним жизненным циклом.

Главная ценность проекта — вертикальная интеграция. Например, пробоина — это не только картинка:

1. попадание создаёт событие повреждения;
2. повреждение меняет состояние панели или runtime-меша;
3. пробоина создаёт приток воды или утечку атмосферы;
4. помещение меняет параметры среды;
5. HUD и отладчик показывают последствия;
6. Utility AI может выбрать ремонт;
7. StateSystem строит исполнение задачи;
8. персонаж идёт по NavMesh и завершает ремонт;
9. состояние попадает в scene save.

Именно такие цепочки превращают отдельные технологии в движок.

1.1. Что реализовано сейчас

Область	Реальное состояние на срезе
Платформа	Windows x64, Visual Studio 2022, toolset v143
Язык	C++20 в Debug x64; конфигурация Release x64 настроена неполно
Графика	DirectX 11 forward renderer, PBR, тени, HDR sky, reflection/refraction, частицы, debug draw
Мир	ECS на `Entity = uint32_t` и типизированных component pools
Физика	Bullet rigid bodies, raycast, fixed step 60 Гц, capsule character controller
Навигация	Recast build и Detour pathfinding, локальное избегание динамических препятствий
Редактор	ImGui, picking, gizmo, инспектор компонентов, импорт и save/load
Контент	Assimp/TinyGLTF, GLB-корабль, материалы, skinned animation

Область	Реальное состояние на срезе
Симуляция	комнаты, двери, кислород, давление, затопление, плавучесть, power grid, damage/repair
Параллельность	JobSystem, CPU-фаза фоновой загрузки, main-thread GPU upload
Тесты	есть `DXFrameworkCoreTests` и локальный build/shader gate; внешний CI пока не настроен

1.2. Уровень зрелости

Точный диагноз: **функционально богатый alpha-прототип собственного движка**.

Это выше «учебной демки», потому что подсистемы реально связаны. Это ещё не production engine, потому что:

- Debug x64 собирается, а Release x64 не воспроизводится из текущего project file;
- есть начальные core-contract tests, но coverage ещё узкое и внешний CI не настроен;
- есть несколько разных соглашений об углах и времени;
- крупные координаторы выросли до нескольких тысяч строк;
- часть физической и графической математики содержит подтверждённые расхождения;
- save/load не является транзакционным runtime-save;
- жизненный цикл ряда raw-ресурсов зависит от ручного порядка очистки.

Это нормальное состояние для быстро выросшего авторского движка. Следующий этап — не добавлять ещё десять подсистем, а сделать существующие правила явными, проверяемыми и повторяемыми.

2. Как пользоваться книгой после долгого перерыва

Не нужно читать её подряд. Сначала установи тип задачи.

Если нужно…	Открыть
вспомнить общий поток кадра	главы 6–7
найти владельца объекта или GPU/Bullet-ресурса	главы 5 и 9
понять Transform, камеру или picking	главы 10–13
разобраться с PBR, HDR или тенями	главы 14–17
исправить дрожание и зависимость от FPS	главы 7, 18–20
понять затопление, давление и плавучесть	главы 21–23
добавить компонент или систему	главы 8 и 34
добавить задачу NPC	главы 27–29

Если нужно…	Открыть
понять scene save и Entity remap	глава 31
найти известный риск	глава 36

После перерыва ответь на семь вопросов:

1. Какие данные являются источником истины?
2. Какая система имеет право менять эти данные?
3. На каком потоке она работает?
4. В какой фазе кадра она вызывается?
5. В каких единицах хранятся значения?
6. Кто владеет внешним ресурсом и кто его освобождает?
7. Какая проверка доказывает корректность?

Если хотя бы один ответ неизвестен, изменение пока опасно.

3. Как движок вырос: история по локальному git

История коммитов показывает естественную эволюцию проекта.

Период	Этап
4–7 марта 2026	DirectX 11 каркас, multitexture, normal map, render-to-texture, ShaderManager, fog
20 марта	добавлены ECS и набор шейдеров
1 апреля	появились collision components
19–25 апреля	первый цикл скелетной анимации и animated mesh
12 мая	отдельный проход по памяти и владению ресурсами
13–17 мая	skybox cache, камера, Editor/Play, player controller, PBR GLB, reflection, animation state, debug UI
26 мая – 14 июня	интеграционные исправления; последний коммит основной ветки `f31cfbb`
после 14 июня	большой незакоммиченный этап: перенос кода в `src`, NPC/NavMesh, корабль, power, atmosphere, particles, repair, асинхронные системы и новые документы

История объясняет нынешнюю форму архитектуры. `ApplicationClass` и `SceneManager` стали большими не из-за одной ошибки, а потому что каждый новый вертикальный срез естественно подключался к уже существующим координаторам. Теперь граница роста достигнута: дальнейшее качество зависит от выделения стабильных интерфейсов и тестов.

4. Карта репозитория

Рабочий проект находится в:

```
C:\Users\Lakis\Desktop\DX_FrameWork\DX_FrameWork
```

Решение Visual Studio находится уровнем выше:

```
C:\Users\Lakis\Desktop\DX_FrameWork\DX_FrameWork.sln
```

Основная документация находится в:

```
C:\Users\Lakis\Desktop\DX_FrameWork\DX_FrameWork\docs
```

4.1. Ответственность каталогов

Каталог	Что является его задачей
`src/Core`	WinMain, окно, приложение, время, DirectX/Bullet bootstrap, JobSystem, файлы
`src/ECS`	registry, pools, базовые компоненты и физико-сценовые системы
`src/Scene`	создание сущностей, импорт, model cache, render traversal, JSON save/load
`src/Render`	D3D11, camera, renderer, models, materials, shaders, textures, culling, text
`src/Gameplay/Player`	input intent, movement, facing, capsule controller, vitals, first person
`src/Gameplay/AI`	Recast/Detour NavMesh и path queries
`src/Gameplay/Ship`	rooms, atmosphere, alerts, room culling, decompression, submarine data
`src/Gameplay/Power`	generators, consumers, battery и распределение мощности
`src/Gameplay/Combat`	damage events, weapons, explosions
`src/Gameplay/Repair`	repair state и runtime mesh restoration
`src/Gameplay/Effects`	CPU particles и particle renderer
`src/Editor`	ImGui panels, gizmo, drag-and-drop, object debug
`assets`	модели, текстуры, HLSL и данные мира
`docs`	руководства, теория и планы

Некоторые новые файлы AI и world time пока находятся в корне проекта: `NPCComponents.h`, `NpcCommandSystem.h`, `NpcUtilitySystem.h`, `StateSystem.h`, `GameTimer.h`, `WorldEvent.h`, `WorldEventLog.h`. Это факт текущего среза, а не рекомендуемое окончательное размещение.

4.2. Горячие точки

Самые крупные авторские файлы на момент аудита:

Файл	Примерный размер	Почему важен

Файл	Примерный размер	Почему важен
src/Editor/EditorUI.cpp	4,8 тыс. строк	почти весь редактор и множество component panels
src/Scene/SceneManager.cpp	4,4–4,9 тыс. строк	импорт, сцена, save/load и часть render traversal
src/Core/ApplicationClass.cpp	2,3 тыс. строк	bootstrap и точный порядок кадра
src/ECS/System/MeshCuttingTest.cpp	2,2 тыс. строк	cutting, damage history и асинхронные операции
StateSystem.h	1,8 тыс. строк	исполнение NPC-состояний и маршрутов
src/Render/Model/assimpmodelclass.cpp	1,8 тыс. строк	импорт, GPU buffers, animation, bone matrices
src/Gameplay/AI/NpcNavMeshSystem.cpp	1,4 тыс. строк	build/query/debug Recast/Detour
src/ECS/System/PhysicsSystem.cpp	1,2 тыс. строк	shapes, Bullet sync, forces, buoyancy

Большой файл не обязательно плох. Риск появляется, когда он одновременно владеет ресурсами, принимает архитектурные решения и выполняет детали нескольких подсистем.

5. Владение и жизненный цикл приложения

5.1. Верхний уровень

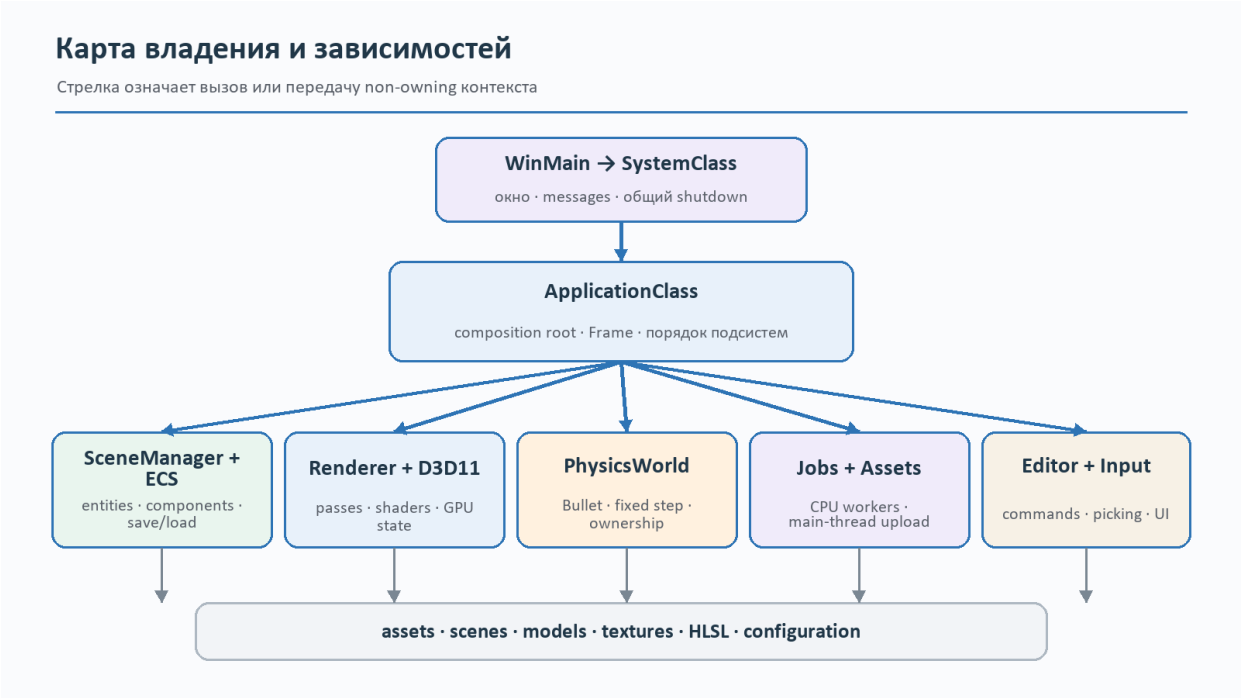


Рисунок 1. Владение и главные зависимости движка.

```
WinMain
  └─ SystemClass (stack)
```

```
    ├─ InputClass (unique_ptr)
    └─ ApplicationClass (raw owning pointer)
        ├─ D3DClass (unique_ptr)
        ├─ CameraClass (unique_ptr)
        ├─ SceneManager (value member)
        │   └─ ECSRegistry (value member)
        ├─ Renderer (raw owning pointer)
        ├─ JobSystem (value member)
        ├─ AssetLoadSystem (value member, depends on JobSystem)
        ├─ EditorUI (value member)
        └─ gameplay/render helper systems
```

`WinMain` создаёт `SystemClass` на стеке, вызывает `Initialize`, затем `Run`, затем `Shutdown`. Это хорошая корневая форма: даже при неуспешной инициализации вызывается общий shutdown.

5.2. Почему порядок shutdown важен

Внешние API хранят невладеющие ссылки:

- Bullet world помнит rigid bodies, ghost objects, actions и constraints;
- D3D context помнит связанные SRV, buffers, shaders и render targets;
- ECS-компоненты могут содержать указатели на Bullet и render resources;
- фоновые jobs могут держать `this`, Entity или CPU-результат для main-thread upload;
- материалы ссылаются на шейдеры ShaderManager.

Поэтому безопасный порядок выглядит так:

1. запретить новые фоновые задачи;
2. дождаться workers и выполнить разрешённые main-thread completions;
3. отвязать D3D state;
4. удалить Bullet actions/constraints/bodies;
5. очистить ECS и scene-owned materials/models;
6. удалить renderer helpers и render textures;
7. закрыть Editor/DirectWrite;
8. удалить ShaderManager;
9. освободить D3D device последним.

> **ОГРАНИЧЕНИЕ.** Текущий `SceneManager::LoadScene` напрямую заменяет `ECSRegistry`, тогда как character controllers и constraints требуют явного удаления из Bullet world. Полный application shutdown учитывает world лучше, чем горячая загрузка новой сцены.

5.3. Правила raw pointer

В проекте raw pointer имеет три разных смысла:

Вид	Пример	Правило
owning	`Renderer* m_renderer`	ровно один `delete`, поле сразу `nullptr`
borrowed	`ShaderBaseClass*` из ShaderManager	никогда не удалять в потребителе

Вид	Пример	Правило
API handle	`ID3D11Buffer*`, `btRigidBody*`	освобождать по контракту API и сначала отвязать от владельца

Если из объявления нельзя понять вид указателя, это будущая ошибка. Целевая модель: owning C++-объекты — `unique_ptr`; COM — `Microsoft::WRL::ComPtr`; невладеющие ссылки — pointer/reference с комментарием и ограниченным временем жизни.

6. Сборка и запуск

6.1. Подтверждённый baseline

`Debug|x64` собирается локальным MSBuild из Visual Studio 2022. Результат находится в:

```
C:\Users\Lakis\Desktop\DX_FrameWork\x64\Debug\DX_FrameWork.exe
```

Использованная команда:

```
& "C:\Program Files\Microsoft Visual Studio\2022\Community\MSBuild\Current\Bin\MSBuild.exe" `
  "C:\Users\Lakis\Desktop\DX_FrameWork\DX_FrameWork.sln" `
  /m /p:Configuration=Debug /p:Platform=x64
```

6.2. Почему Release x64 сейчас падает

В `DX_FrameWork.vcxproj` Debug x64 получает:

- `EngineIncludeDirectories`;
- Bullet include paths;
- C++20;
- Assimp, Bullet Debug, Recast/Detour libraries;
- Windows subsystem.

Release x64 содержит Recast/Detour, но не повторяет include directories, C++20 и полный набор Assimp/Bullet libraries. Поэтому это конфигурационный дефект, а не доказательство ошибок каждой исходной подсистемы.

6.3. Runtime-зависимость от working directory

Многие пути записаны как `assets/...`. `ApplicationClass` создаёт файловую систему от `current_path()/assets`. Следовательно, запуск из Visual Studio работает, а запуск EXE из произвольной папки может не найти shaders/models/textures. Готовый движок должен вычислять runtime root от пути EXE или получать его явно.

6.4. Минимальный smoke-test

После любой архитектурной правки проверить:

1. запуск без shader compiler error;
2. загрузку сцены и HDR sky;
3. `F5` Editor/Play;
4. ходьбу, бег, прыжок и спокойный idle на полу;
5. NPC path через дверь;
6. пробоину и изменение среды;
7. ремонт;
8. save → очистка → load;
9. возврат в Editor;
10. чистый shutdown без live D3D objects и dangling Bullet objects.

7. Один кадр: фактический timeline

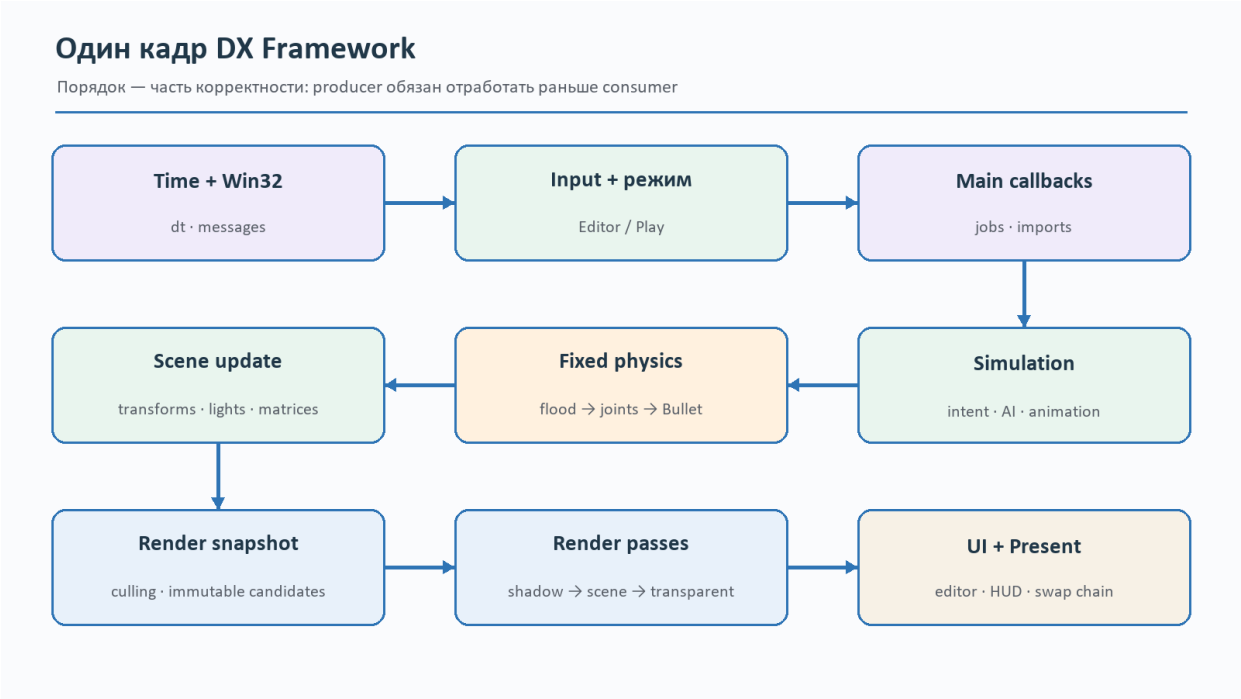


Рисунок 2. Фактическая последовательность фаз кадра.

Главная точка входа — `ApplicationClass::Frame(float dt)`. Порядок систем является частью поведения, а не косметикой.

```
Windows messages / TimeManager
        ↓
InputManager.Update + EventQueue.Clear
        ↓
Engine hotkeys (F1/F2/F5/F6)
```

```
        ↓
Main-thread jobs + asset upload + async ship import budget
        ↓
Editor camera  [Editor]
        ↓
Interaction → doors → weapon/repair  [Play]
        ↓
Player intent
        ↓
NPC Utility → StateSystem → Facing → CharacterController → Animation
        ↓
Damage → particles
        ↓
Sync compartments
        ↓
Flooding → joints → submarine → PhysicsSystem → pressure
        ↓
Post-physics camera
        ↓
SceneManager.UpdateAll
  animation / attachments / rooms / generator / grid / lights /
  oxygen / room environment / player vitals / ship alerts
        ↓
Camera matrices + debug/navmesh
        ↓
Renderer.RenderFrame
        ↓
Editor UI or Play UI / HUD
        ↓
Present
```

7.1. Зависимости порядка

- Utility должна принять решение до `StateSystem`, иначе команда исполнится на кадр позже.
- Facing читает `walkDirection`, поэтому идёт после AI, но до controller/animation.
- Камера Play следует после физики, иначе будет показывать предыдущую позицию.
- Power grid должен обновиться до потребителей и oxygen generator.
- Render видит только уже завершённое состояние кадра.

7.2. Editor и Play

`F5` переключает режим. При выходе из Play код явно сбрасывает movement intent и Bullet controller velocity, иначе кэшированное направление продолжило бы двигать персонажа в Editor. Это хороший пример архитектурного инварианта: **отключить producer недостаточно; нужно сбросить состояние consumer**.

7.3. Очередь Windows

`SystemClass::Run` использует `PeekMessage`, но на срезе обрабатывает одно сообщение за итерацию. При потоке mouse/window messages очередь может отставать. Целевая форма — внутренний `while (PeekMessage(...))`, после которого выполняется ровно один simulation/render frame.

8. Время и численная устойчивость

В проекте существуют два понятия времени:

- `TimeManager` — QPC, pause/step и delta игрового цикла;
- `GameTimer` — новый world/game total time для событий.

Большинство систем получает `dt` напрямую. Такое дублирование допустимо временно, но без контракта непонятно, что останавливается при pause и что масштабируется.

8.1. Variable step

Обычное интегрирование:

```
x_next = x + v · dt
v_next = v + a · dt
```

Оно подходит для камеры, UI и эффектов, если `dt` ограничен. После breakpoint или подвисания `dt` может стать огромным, поэтому безопасно:

```
dt_sim = clamp(dt_real, 0, dt_max)
```

8.2. Fixed step Bullet

`PhysicsWorld` использует:

```
h = 1 / 60 s
accumulator = min(accumulator + dt, 0.25 s)
while accumulator >= h:
    stepSimulation(h)
    accumulator -= h
alpha = accumulator / h
```

Ограничение 0,25 секунды предотвращает «спираль смерти»: движок не пытается бесконечно догнать потерянное время.

8.3. Интерполяция

После fixed tick визуальная позиция может быть:

```
x_render = lerp(x_previous, x_current, alpha)
```

В проекте character controller имеет отдельную стабилизацию/интерполяцию, а обычные rigid bodies в основном копируют текущий Bullet transform. Поэтому на высоком FPS часть тел может визуально обновляться ступенями 60 Гц.

8.4. FPS-независимое сглаживание

Формула `lerp(old, target, k · dt)` лишь приближённо независима от FPS и должна ограничивать `t` в `[0,1]`. Более устойчивый коэффициент экспоненциального приближения:

```
t = 1 - exp(-lambda · dt)
value = lerp(value, target, t)
```

Здесь `lambda` — скорость сходимости в 1/с. Период полураспада связан с ней:

```
lambda = ln(2) / halfLife
```

> **ОГРАНИЧЕНИЕ.** Сглаживание submersion в `PhysicsSystem` использует постоянные `0.3` на render frame. Это означает разную плавучесть при 30, 60 и 144 FPS.

8.5. Силы должны жить внутри fixed tick

На срезе buoyancy/drag/torque применяются один раз на render frame, после чего Bullet может выполнить ноль, один или несколько fixed steps. При низком FPS Bullet очистит силы после первого шага, и следующие шаги останутся без них; при высоком FPS силы могут накапливаться до следующего шага. Целевая модель: вычислять continuous forces для каждого fixed tick.

Часть II. C++ и ECS именно в этом проекте

9. C++: значения, ссылки, указатели и RAII

9.1. Значение

```
TransformComponent transform;
```

Объект живёт в своей области видимости. Для ECS-компонента это обычно означает хранение внутри `unordered_map<Entity, T>`.

9.2. Ссылка

```
void Update(ECSRegistry& registry, float dt);
```

Ссылка обязательна и не бывает `nullptr`. Она не передаёт владение.

9.3. Указатель

```
TransformComponent* transform = registry.GetComponent<TransformComponent>(entity);
if (!transform)
    return;
```

Указатель выбран, потому что компонент может отсутствовать. Проверка `nullptr` — часть контракта.

9.4. RAII

RAII связывает ресурс со временем жизни объекта. В движке это особенно важно для:

- COM resources (`ID3D11Buffer`, `ID3D11ShaderResourceView`);
- Bullet body/shape/motion state;
- worker threads;
- файлов;
- временных GPU states.

`RigidBodyComponent` пытается реализовать RAII и запрещает копирование. Это правильное направление. Но move-конструктор обязан переносить **каждое** поле владения; пропущенный raw pointer создаёт leak или double-delete.

9.5. Rule of zero / five

Если класс владеет ресурсом, есть два безопасных пути:

1. хранить ресурс в готовом RAII-типе и использовать rule of zero;
2. явно реализовать destructor, move constructor и move assignment, а copy удалить.

Пустой copy constructor хуже удалённого: он компилируется и создаёт объект с неинициализированным состоянием. На срезе такие конструкторы есть у некоторых legacy-классов, включая камеру/D3D/SystemClass.

10. Как устроена ECS

10.1. Entity

```
using Entity = std::uint32_t;
```

Registry выдаёт монотонно растущий ID. Entity не содержит данных; это ключ для component pools.

10.2. Component pool

Для обычного `T`:

```
ComponentManager<T>
  └ unordered_map<Entity, T>
```

Для `ActiveComponent` используется `unordered_set<Entity>`, потому что tag не хранит полезных полей.

10.3. Type erasure

Registry хранит разные pools в одной таблице:

```
unordered_map<type_index, unique_ptr<IComponentPool>>
```

`IComponentPool::Remove(Entity)` позволяет `DestroyEntity` удалить ID из каждого пула без знания конкретного `T`.

10.4. Добавление компонента

Есть copy-версия для копируемых типов и move-версия для owning-компонентов:

```
registry.AddComponent<TransformComponent>(entity, transform);
registry.AddComponent<RigidBodyComponent>(entity, std::move(body));
```

Для raw-resource компонента второй вариант обязателен.

10.5. Сложность

Операция	Ожидаемая сложность
`Get/Has/Add/Remove` одного компонента	O(1) average
`DestroyEntity`	O(number of component types)
`ForEach<T>`	O(count(T))
`ForEach<T1,T2>`	O(count(T1)) с lookup во втором pool

10.6. Подтверждённые ограничения

1. Нет generation counter. После будущего reuse старый Entity сможет указывать на новый объект.
2. Нет явного alive set; `DestroyEntity` только удаляет компоненты.
3. `GetComponent` и `HasComponent` создают пустой pool при первом чтении отсутствующего типа.
4. `ForEach` идёт напрямую по `unordered_map`; добавление/удаление того же типа из callback может инвалидировать iterator.
5. Нет deferred command buffer для структурных изменений.

Целевая Entity:

```
64 bits = generation | index
```

А структурные изменения должны накапливаться в command buffer и применяться после system phase.

11. Как добавить компонент и систему

11.1. Компонент — только данные

```
struct HeatComponent
{
    float kelvin = 293.15f;
    float heatCapacityJPerK = 1000.0f;
};
```

У полей должны быть единицы, диапазон и значение по умолчанию.

11.2. Система — одно правило изменения

```
class HeatSystem
{
public:
    static void Update(ECSRegistry& registry, float dt)
    {
        registry.ForEach<HeatComponent>(
            [dt](Entity, HeatComponent& heat)
            {
                heat.kelvin = std::max(0.0f, heat.kelvin - 0.2f * dt);
            });
    }
};
```

11.3. До подключения определить

- phase кадра;
- scaled или unscaled time;
- main thread или worker;
- источник истины;
- сериализацию;
- debug representation;
- минимум один детерминированный тест.

Часть III. Математический фундамент

12. Единицы и координатные соглашения

12.1. Базис

Движок использует left-handed DirectX world:

```
+X = вправо
+Y = вверх
+Z = вперёд
```

Проекция DirectX использует NDC depth `[0,1]`.

12.2. Единицы

Величина	Контракт
позиция/расстояние	метры по смыслу Bullet/Recast
время	секунды
скорость	м/с
ускорение	м/с²
масса	кг

Величина	Контракт
сила	ньютон
давление PressureSystem	паскаль
Transform rotation	радианы
Camera rotation	градусы, конвертируются перед DirectXMath
spotlight angles	градусы
HLSL colors	linear до tone mapping, если pipeline соблюдён

> **КРИТИЧЕСКОЕ ИСПРАВЛЕНИЕ СТАРОЙ ДОКУМЕНТАЦИИ.** `TransformComponent::rotation = {0, 90, 0}` не означает 90 градусов. Компонент передаёт значения прямо в `XMMatrixRotationX/Y/Z`, поэтому 90° нужно записать как `XMConvertToRadians(90.0f)` или примерно `1.5708f`.

12.3. Контракт, которого пока не хватает

Тип должен сообщать единицу именем или API:

```
Radians yaw;
Degrees editorYaw;
Meters distance;
Seconds dt;
```

Даже без strong types полезны суффиксы: `yawRadians`, `outerAngleDegrees`, `batteryChargeMwh`.

13. Векторы

13.1. Длина и нормализация

Для `v = (x,y,z)`:

```
|v| = sqrt(x² + y² + z²)
normalize(v) = v / |v|
```

Нельзя нормализовать почти нулевой вектор без проверки. Иначе появится NaN, который затем загрязнит Transform, Bullet и shader constants.

13.2. Dot product

```
dot(a,b) = ax·bx + ay·by + az·bz = |a||b|cos(theta)
```

Применения:

- находится ли поверхность под персонажем: `dot(normal, up)`;
- освещение Lambert: `max(dot(N,L),0)`;
- Fresnel/BRDF: `N·V`, `N·L`, `N·H`;
- направление spot light;
- фронт/тыл объекта.

13.3. Cross product

```
cross(a,b)
```

Даёт вектор, перпендикулярный обоим. Применения: basis tangent space, normal triangle, torque arm, camera right. Порядок аргументов меняет знак.

13.4. Camera-relative movement

При yaw камеры в радианах:

```
forward = (sin(yaw), 0, cos(yaw))
right   = (cos(yaw), 0, -sin(yaw))
wish    = inputForward·forward + inputRight·right
```

После сложения `wish` нормализуется, иначе диагональ движется в `sqrt(2)` раза быстрее.

14. Матрицы и Transform

14.1. Текущая world matrix

`TransformComponent::GetWorldMatrix` строит:

```
W = S · Rx · Ry · Rz · T
```

Проект использует row-vector форму в HLSL:

```
output.position = mul(input.position, worldMatrix);
output.position = mul(output.position, viewMatrix);
output.position = mul(output.position, projectionMatrix);
```

Следовательно:

```
p_clip = p_local · W · V · P
```

14.2. Пространства

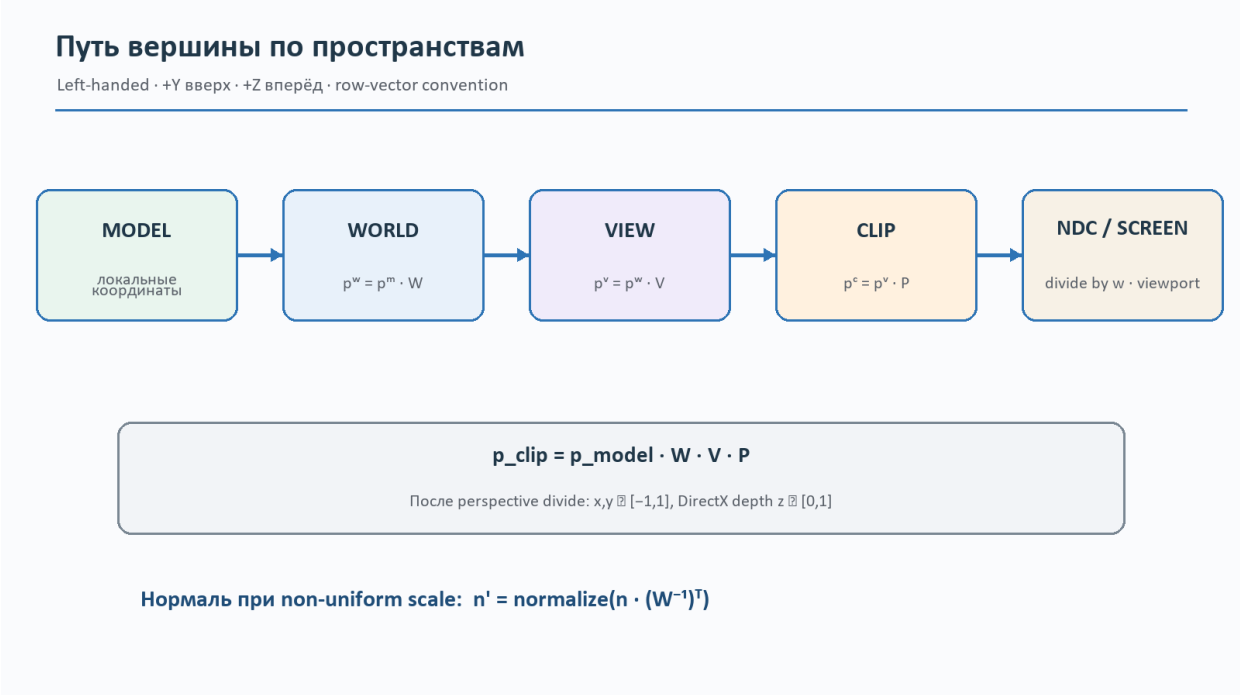


Рисунок 3. Преобразование вершины из model space в экран.

```
local/model → world → view/camera → clip → NDC → screen
```

- local: вершина относительно pivot модели;
- world: объект в мире;
- view: мир относительно камеры;
- clip: до homogeneous divide;
- NDC: после деления на `w`;
- screen: пиксели viewport.

14.3. CPU/HLSL transpose

DirectXMath и layout constant buffers требуют явного согласования. Обычные W/V/P matrices CPU транспонирует перед записью в buffer. Bone matrices — исключение: HLSL объявляет их `row_major`, и код загружает без transpose. Любое изменение здесь должно проверяться на одном тестовом треугольнике и одной кости, а не «на глаз» на сложной модели.

14.4. Normal matrix

Нормаль нельзя всегда умножать на обычную `3×3` world matrix. При non-uniform scale правильно:

```
N_world = normalize(N_local · transpose(inverse(W3x3)))
```

На срезе PBR/light/normalmap/terrain vertex shaders используют `W3x3` напрямую. При масштабе `(2,1,0.5)` освещение становится геометрически неверным.

14.5. Pivot и collider center

Bounds модели имеют `min` и `max`:

```
halfExtents = (max - min) / 2
center      = (max + min) / 2
```

Если Bullet shape получает только half extents, но body origin остаётся в pivot, collider смещается для модели с нецентральным pivot. Текущий box/sphere path не везде учитывает `center`.

15. Euler angles и quaternion

Euler удобен редактору, quaternion — интерполяции и физике.

15.1. Почему порядок важен

Повороты не коммутируют:

```
Rx · Ry != Ry · Rx
```

Transform строит `Rx·Ry·Rz`, а Bullet helper использует `XMQuaternionRotationRollPitchYaw`, эквивалентный другому порядку составных поворотов. При одной активной оси расхождение незаметно; при одновременном pitch/yaw/roll визуальная модель и collider могут смотреть по-разному.

15.2. Quaternion

```
q = (x,y,z,w), |q| = 1
```

Для интерполяции используется SLERP:

```
q(t) = slerp(q0, q1, t)
```

SLERP идёт по кратчайшей дуге единичной 4D-сферы и не создаёт типичных скачков Euler interpolation.

15.3. Целевая модель

- runtime orientation хранится как normalized quaternion;
- editor Euler — только представление;
- одна библиотечная функция строит matrix и basis;
- render, Bullet, animation attachments и save/load используют одну конвенцию;
- тест перебирает составные углы и сравнивает forward/up в render и physics.

16. Камера и проекция

16.1. View matrix

Камера использует `XMMatrixLookAtLH`:

```
V = LookAtLH(position, position + forward, up)
```

При нулевом roll направление камеры соответствует:

```
Fx = cos(pitch) · sin(yaw)
Fy = -sin(pitch)
Fz = cos(pitch) · cos(yaw)
```

`CameraClass` хранит pitch/yaw/roll в градусах и конвертирует их в радианы перед DirectXMath.

16.2. Third-person camera

```
C = target - distance · forward
```

Покомпонентно:

```
Cx = Tx - d·sin(yaw)·cos(pitch)
Cy = Ty + d·sin(pitch)
Cz = Tz - d·cos(yaw)·cos(pitch)
```

16.3. Perspective projection

Основная камера:

- vertical FOV: 75°;
- near: 0.3;
- far: 1000;
- aspect: width/height;
- `XMMatrixPerspectiveFovLH`.

Если `f = 1/tan(FOV/2)`:

```
x_clip = x · f/aspect
y_clip = y · f
```

Малый near улучшает видимость рядом с камерой, но ухудшает распределение точности depth. Отношение `far/near` здесь примерно 3333, приемлемое, но first-person pass использует отдельную более близкую near plane.

16.4. Picking ray

Экранная мышь переводится в NDC, затем far-point проходит обратное преобразование:

```
p_world = homogeneousDivide(p_ndc · inverse(VP))
origin  = cameraPosition
dir     = normalize(p_world - origin)
```

Этот ray используется редактором, взаимодействием, оружием и F6 mesh cutting. Значит picking — общий engine service, и изменение его конвенций должно тестироваться всеми четырьмя потребителями.

17. Bounds и frustum culling

17.1. AABB и sphere

Sphere дешевле и нечувствительна к вращению, но отсекает грубее. AABB точнее для осево ориентированных bounds, но после вращения нужно построить новый world AABB из всех углов или использовать conservative transform.

17.2. Frustum

Frustum извлекается из view-projection и проверяет bounds кандидатов. `RenderCandidateSystem` делает важную вещь правильно: ECS snapshot создаётся на main thread, а worker получает независимые данные, поэтому не читает живые `unordered_map` во время структурных изменений.

17.3. Ограничение cache

Асинхронный результат принимается, если элементы view/projection отличаются не больше эвристического порога. Это повышает производительность, но может временно использовать устаревший список. Проверка: быстро двигать камеру через тонкую стену и искать однокадровые пропадания.

Часть IV. DirectX 11 и рендер

18. Ресурсы и состояния DirectX 11

DirectX 11 — stateful API. Результат draw call зависит не только от переданных вершин, но и от всего состояния context: render targets, depth, blend, rasterizer, shaders, buffers, SRV и samplers.

18.1. Базовые ресурсы

Ресурс	Роль
`IDXGISwapChain`	очередь backbuffers и Present
`ID3D11RenderTargetView`	поверхность, куда пишет pixel shader
`ID3D11DepthStencilView`	depth/stencil testing

Ресурс	Роль
`ID3D11Buffer`	vertices, indices, constants
`ID3D11ShaderResourceView`	чтение texture/buffer в shader
`ID3D11SamplerState`	фильтрация и addressing
rasterizer state	culling, fill, bias, scissor
depth-stencil state	depth compare/write и stencil
blend state	смешивание результата pixel shader

Текущий backbuffer — `R8G8B8A8_UNORM`, depth — `D24_UNORM_S8_UINT`, MSAA выключен, основной rasterizer использует `CULL_NONE`.

18.2. Constant buffer alignment

HLSL constant buffer упаковывается блоками по 16 байт. CPU-структура должна совпадать по размеру и layout. Ошибка «создали 64 байта, записали 144» не обязана сразу упасть: она повреждает mapped memory и проявляется случайно.

> **ОГРАНИЧЕНИЕ.** В `SkyShaderClass` существует неиспользуемый legacy-метод, который может записать структуру больше созданного matrix buffer. Текущий рабочий path пишет только матрицу, поэтому дефект латентный.

18.3. COM lifetime

COM-объект освобождается `Release`, но ручной код уязвим к раннему `return`. Целевая форма:

```
Microsoft::WRL::ComPtr<ID3D11Buffer> buffer;
```

`ComPtr` автоматически балансирует reference count и делает partial initialization безопаснее.

18.4. Полностью заданное состояние

Каждый render pass должен явно отвечать:

1. какой RTV/DSV связан;
2. какой viewport;
3. depth test и depth write;
4. blend;
5. culling/fill;
6. какие SRV и samplers заняты;
7. какие slots очищаются после pass.

Иначе следующий pass наследует невидимый остаток предыдущего.

19. Фактический render pipeline

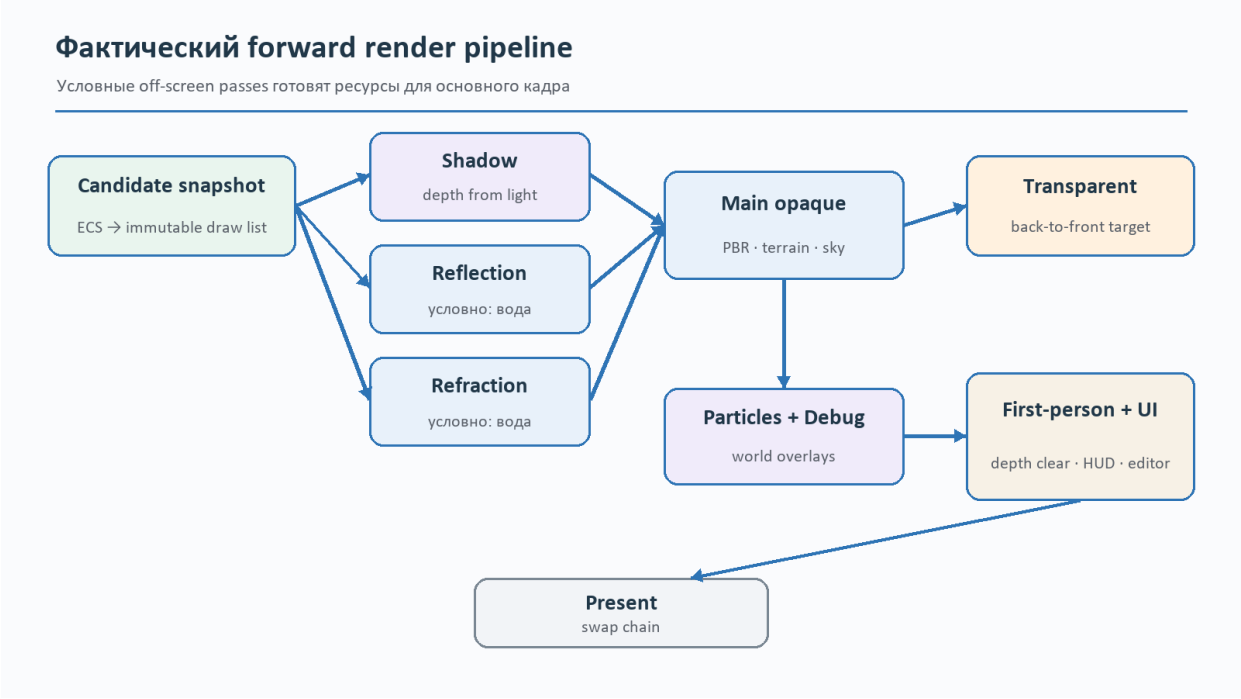


Рисунок 4. Основные проходы forward-рендера.

`Renderer::RenderFrame` реализует forward renderer. G-buffer/deferred lighting в проекте нет.

```
RenderCandidate snapshot/request
  ↓
Sun + environment update
  ↓
Directional shadow map
  ↓
Reflection texture (если нужна)
  ↓
Refraction texture (если нужна)
  ↓
Editor preview (если нужен)
  ↓
Backbuffer clear + equirectangular sky
  ↓
Opaque scene + grass
  ↓
World sprites
  ↓
Transparent scene
  ↓
Particles
  ↓
Debug geometry
  ↓
Depth clear + first-person geometry
  ↓
UI/HUD + Present
```

19.1. Почему first-person рисуется отдельно

Оружие и руки не должны проваливаться в стены мира. Отдельный pass после очистки depth и с собственной projection делает view model визуально стабильной. Цена — нужны отдельные

правила shadows, FOV и clipping; объект first-person нельзя случайно отрисовать ещё раз в world pass.

19.2. Candidate culling

Main thread копирует минимальные render-данные из ECS, worker проверяет frustum, main thread использует только подходящий по camera matrices результат. Это хороший пример безопасной параллельности: worker не трогает D3D immediate context и не обходит живой ECS.

19.3. Room culling

Кроме frustum, `RoomCullingSystem` определяет видимые помещения через связи комнат/дверей. Это portal-like gameplay culling. Оно не должно влиять на саму симуляцию: закрытая за дверью комната может не рисоваться, но oxygen/power/NPC продолжают обновляться.

19.4. Draw context как целевой интерфейс

Сейчас функции получают длинные наборы matrices, arrays и pointers. Безопаснее один объект:

```
struct RenderViewContext
{
    XMMATRIX view;
    XMMATRIX projection;
    XMFLOAT3 cameraPosition;
    LightPacket lights;
    FogPacket fog;
    RenderPassKind pass;
};
```

Так компилятор и validation layer не позволят refraction забыть camera position или light packet.

20. PBR: математика Cook–Torrance

Основной `pbr.ps` реализует metallic-roughness Cook–Torrance с GGX, Fresnel-Schlick и Smith geometry.

20.1. Векторы

```
N = surface normal
V = direction surface → camera
L = direction surface → light
H = normalize(V + L)
```

Все dot products ограничиваются снизу нулём:

```
NdotL = max(dot(N,L), 0)
NdotV = max(dot(N,V), epsilon)
```

20.2. Roughness и GGX distribution

Код использует:

```
a = roughness²
D_GGX = a² / (π · ((N·H)²(a² - 1) + 1)²)
```

`D` описывает долю микрограней, ориентированных как `H`. При малой roughness пик узкий; при большой — широкий.

20.3. Fresnel-Schlick

```
F = F0 + (1 - F0)(1 - cosTheta)⁵
F0 = lerp(0.04, albedo, metalness)
```

Диэлектрик отражает около 4% при прямом взгляде; металл берёт цвет отражения из albedo.

20.4. Smith/Schlick-GGX geometry

```
k = (roughness + 1)² / 8
G1(x) = x / (x(1-k) + k)
G = G1(N·V) · G1(N·L)
```

`G` учитывает masking/shadowing микрограней.

20.5. Specular BRDF

```
specular = D · G · F / max(4(N·V)(N·L), epsilon)
```

Diffuse сохраняет энергию:

```
kS = F
kD = (1 - kS)(1 - metalness)
diffuse = kD · albedo / π
```

Итог одного света:

```
Lo += (diffuse + specular) · radiance · (N·L)
```

20.6. Точечное и spot-затухание

Точечный свет использует художественную модель конечного радиуса:

```
attenuation(d) = (1 - d/R)², если d < R; иначе 0
```

Это не физический `1/d²`, зато свет строго заканчивается на `R` и удобен редактору.

Spot factor:

```
smoothstep(cosOuter, cosInner, coneDirection · L)
```

20.7. Ограничение количества lights

Shader поддерживает одно солнце и четыре point/spot lights.
`SceneManager::GatherLightsForShader` берёт первые четыре активных ECS-источника, а не ближайшие/важнейшие. При пятом источнике результат зависит от порядка `unordered_map` и может «прыгать». Целевая политика: score по влиянию на объект/камеру и стабильная сортировка.

21. Цвет, HDR и IBL

21.1. Linear и sRGB

Физика света выполняется в linear space. Обычная цветовая текстура хранится в sRGB и декодируется при sample; normal/metal/roughness — linear data.

Правильный pipeline:

```
sRGB textures → decode to linear
lighting/blending in linear HDR
exposure + tone mapping
linear → sRGB output
```

21.2. Текущий pipeline

PBR shader выполняет ACES-like tone mapping и gamma внутри каждого draw, а пишет сразу в UNORM backbuffer. Следствия:

- прозрачность смешивается после gamma;
- невозможно корректно применить общий exposure;
- bloom/postprocess не видит исходную HDR-энергию;
- разные shader families могут иметь разные color transforms.

Целевая модель: общий HDR scene target и ровно один финальный postprocess.

21.3. Environment map

Текущий sky — один equirectangular HDR. PBR читает его как environment reflection, но не использует:

- diffuse irradiance convolution;
- prefiltered specular mips;
- BRDF integration LUT.

Roughness поэтому в основном ослабляет отражение, а не физически размывает его.

Split-sum IBL:

```
ambientDiffuse  = irradiance(N) · albedo
ambientSpecular = prefiltered(R, roughness) · (F · brdf.x + brdf.y)
```

21.4. Несовпадение ориентации HDR

Sky shader добавляет к U примерно `+0.25`, а PBR direction→equirect UV — нет. Видимая панорама и отражение повернуты относительно друг друга примерно на 90°. Должна существовать одна общая функция mapping и один rotation parameter.

22. Тени

22.1. Текущая модель

- один directional light;
- orthographic shadow map 1024×1024;
- 2×2 PCF;
- receiver slope bias.

Light matrices:

```
V_light = LookAtLH(center - direction·distance, center, up)
P_light = OrthoLH(size, size, near, far)
```

Receiver bias:

```
bias = max(0.00015, 0.0008 · (1 - N·L))
```

22.2. Подтверждённые ограничения

1. Shadow volume центрирован возле world origin, а не camera frustum.
2. Casters предварительно отсекаются main-camera frustum; объект за камерой может давать видимую тень, но быть удалён.
3. Нет texel snapping — возможен shimmering.
4. Shadow pass не skinning-aware — animated mesh даёт bind-pose shadow.
5. Нет alpha cutout.
6. PCF предполагает фиксированное разрешение.
7. Grass не включён полностью в shadow model.

22.3. Путь развития

Сначала camera-fitted stabilized single cascade. Только после стабильности — CSM:

```
near cascade: высокая детализация
middle: средняя
far: большая площадь
```

23. Прозрачность, reflection и refraction

23.1. Правильная alpha transparency

Для обычного `srcAlpha / (1-srcAlpha)`:

```
opaque:       depth test ON, depth write ON
transparent:  depth test ON, depth write OFF, sort back-to-front
```

На срезе transparent objects не сортируются и продолжают использовать depth-write state. Если модель имеет хотя бы одну прозрачную embedded part, весь объект может уйти в transparent pass, включая непрозрачные части.

23.2. Refraction defect

В refraction path локальные arrays lights объявлены, но не заполняются до передачи в render. Camera position также остаётся default `(0,0,0)`. PBR specular/Fresnel получают случайный или неверный input. Это P0-дефект, потому что Debug/Release могут показывать разные картинки.

23.3. Reflection camera

Для горизонтальной плоскости y=h:

```
C'_y = 2h - C_y
```

View matrix отражается, но PBR должен получить и отражённую camera position. Текущий исключающий scene path теряет входную позицию и передаёт ноль.

23.4. Вода

Текущий water shader смешивает reflection/refraction почти постоянным коэффициентом. Более полная модель использует Fresnel:

```
F = F0 + (1-F0)(1-N·V)⁵
color = lerp(refraction · absorption, reflection, F)
```

Дальше добавляются depth/thickness absorption, normal waves и foam. Но сначала нужно исправить cameras, clipping и render states.

24. Частицы и взрывы

24.1. CPU simulation

Максимум около 4096 частиц. Интегрирование:

```
v_next = (v + a·dt) · exp(-drag·dt)
x_next = x + v_next·dt
```

Emission accumulator:

```
A += rate·dt
spawn = floor(A)
A -= spawn
```

Это сохраняет дробную часть и делает среднюю скорость emission устойчивой к FPS.

24.2. Billboard

Right/up извлекаются из inverse view. На каждую частицу строится quad из шести CPU-вершин. Это просто и понятно, но масштабируется хуже instancing.

24.3. Случайное распределение

Нормализованная точка равномерного куба **не** равномерна на сфере. Корректный способ:

```
z = 1 - 2u
phi = 2πv
dir = (sqrt(1-z²)cos(phi), z, sqrt(1-z²)sin(phi))
```

Для равномерного объёма шара радиус:

```
r = R · cbrt(u)
```

а не `R·u`.

24.4. Сортировка

Renderer сначала сортирует alpha particles far-to-near, но затем группирует их по texture через map. Между разными текстурами глобальный порядок теряется. Нужен либо stable batching с ограничениями, либо OIT.

24.5. Explosion impulse

```
J = J0 · (1 - d/R)²
```

Текущая модель не проверяет line of sight, поэтому импульс проходит через стену. Для engine-level explosion query нужны physics overlap + ray/occlusion и точка приложения силы.

25. Скелетная анимация

25.1. Local и global bone transforms

Position/scale интерполируются линейно, rotation — SLERP. Local matrix:

```
M_local = S · R · T
M_global = M_local · M_parent
```

Skin matrix в row-vector convention:

```
M_skin = M_offset · M_animatedGlobal
```

25.2. Vertex blending

До четырёх bone influences:

```
p_skinned = Σ weight_i · (p · M_skin_i)
```

Weights нормализуются importer-ом.

25.3. Ограничения

- HLSL buffer рассчитан на 128 bones;
- importer не всегда выдаёт явную ошибку при превышении;
- bone ID ≥128 пропускается;
- shadow/refraction/transparent passes не везде получают current bone buffer;
- в `assimpmodelclass.cpp` остаётся legacy hierarchy path с другим порядком matrices.

Проверка должна одновременно показывать персонажа напрямую, в reflection/refraction и shadow.

Часть V. Bullet и симуляция мира

26. Bullet Physics и синхронизация

26.1. World settings

```
gravity = (0, -9.81, 0) m/s²
fixed step = 1/60 s
solver iterations = 50
split impulse = enabled
ERP ≈ 0.2
```

26.2. Типы тел

Тип	Кто задаёт transform
static	создаётся один раз; мир не двигает
kinematic	ECS/editor задаёт каждый кадр, Bullet только сталкивает
dynamic	Bullet — источник истины; ECS получает результат

Нельзя одновременно каждый кадр телепортировать dynamic Transform из ECS и ожидать честную физику. В проекте есть детектор внешнего изменения: editor teleport обновляет Bullet и сбрасывает velocities.

26.3. Collision shapes

Поддерживаются box, sphere, cylinder, capsule, plane, convex hull, triangle/GImpact mesh, compound, StairRamp и объявленный VHACD.

Правила:

- static concave: triangle mesh допустим;
- dynamic concave: GImpact дорог, convex decomposition предпочтительнее;
- capsule: cylinder height не может быть отрицательной;
- sphere при non-uniform scale либо использует max scale, либо перестаёт быть sphere;
- compound child должен учитывать local center/rotation.

> **ОГРАНИЧЕНИЕ.** VHACD присутствует в enum/editor, но основной path не создаёт полноценную shape во всех ветках. Функция выглядит доступной, хотя физически может отсутствовать.

26.4. Настройки компонента

`RigidBodyComponent` хранит friction, damping, CCD и angular factor, однако create path на срезе часть значений задаёт константами. UI-поле, которое не влияет на runtime, опаснее отсутствующего поля: разработчик верит ложной настройке.

26.5. Interior collider

Код создаёт `interiorBody`, но аудит не нашёл регистрацию `world->addRigidBody(interiorBody, ...)`. Выделенный объект без регистрации не участвует в collision detection.

26.6. Ownership

Безопасный teardown:

```
world.removeConstraint/removeAction/removeRigidBody
→ delete controller/constraint/body motion state
→ delete body
→ delete shapes/triangle meshes
```

Прямой reset Registry опасен для character ghost/action и constraints, потому что их destructors не все снимают registration.

27. Character controller

27.1. Геометрия

Персонаж — вертикальная capsule. Полная высота:

```
H_total = H_cylinder + 2R
```

Центр capsule смещён относительно feet Transform. Поэтому попытка «починить провал» случайным подъёмом всей capsule ломает ступеньки, camera anchor и reach.

27.2. Ground test

Поверхность walkable, если:

```
dot(normal, up) >= cos(maxSlope)
```

27.3. Auto-step

Последовательность sweep:

```
up → forward → down → validate slope
```

Это хорошая engine-функция: один controller используется игроком и NPC, поэтому они сталкиваются с миром одинаково.

27.4. Stabilization

Код содержит post-step ground correction и dead zone, появившиеся из реального бага дрожания. Важный урок: прежде чем добавлять smoothing, определить источник:

- conflict ECS/Bullet ownership;
- contact normal noise;
- capsule center offset;
- camera reading pre-physics pose;
- animation root motion.

27.5. Прыжок

На срезе `wantJump` формируется от удержания клавиши. После приземления удержанная кнопка может запустить новый прыжок. Engine input action должен различать `pressed`, `held`, `released`.

28. Давление, затопление и плавучесть

28.1. Гидростатическое давление

```
P = P0 + ρgh
```

Где:

- `ρ ≈ 1000 kg/m³` для воды;
- `g = 9.81 m/s²`;
- `h` — глубина в метрах;
- результат — Pa.

`PressureSystem` следует этой модели.

28.2. Скорость через пробоину

Из Bernoulli/Torricelli:

```
ΔP = ρgh
```

```
v = sqrt(2ΔP/ρ) = sqrt(2gh)
Q = Cd · A · v
```

Если `size` — радиус:

```
A = πr²
```

28.3. Подтверждённая ошибка единиц

`FloodingSystem` вычисляет:

```
pressure = depth · 9.81
velocity = sqrt(2 · pressure / 1000)
```

В первой строке отсутствует `ρ`, а во второй оно делится ещё раз. Полученная скорость примерно в `sqrt(1000) ≈ 31.6` раза меньше Torricelli.

Пример при `h=10m`:

```
правильно: sqrt(2·9.81·10) ≈ 14.0 m/s
текущий код: sqrt(2·9.81·10/1000) ≈ 0.443 m/s
```

28.4. Water volume

```
V_next = clamp(V + (Q_in - Q_pump)dt, 0, V_room)
waterLevel = (V / V_room) · roomHeight
```

Нужен guard `V_room > epsilon`, иначе division by zero.

28.5. Архимедова сила

```
F_b = ρwater · g · V_displaced
```

Текущая модель:

```
submersion = clamp((waterLevel - bottom)/height, 0, 1)
F_b = ρgV · submersion · multiplier
F_net_y = F_b - ρwater·V_inside·g
```

28.6. Двойной учёт воды

`FloodingSystem` уменьшает `buoy.volume` на внутренний объём воды. `PhysicsSystem` одновременно оставляет отдельный вес внутренней воды. Физически затопление должно либо:

- уменьшать эффективную герметичную displacement volume;
- либо добавлять массу воды;
- либо использовать подробно моделируемый flooded volume,

но не применять упрощённые эффекты дважды без вывода. На срезе потеря плавучести и дополнительный вес суммируются.

28.7. Центр плавучести

Сила сейчас прикладывается в центре массы, поэтому не создаёт восстанавливающий момент. Более выразительная модель использует несколько sample points или вычисленный center of buoyancy:

```
τ = (centerBuoyancy - centerMass) × F_b
```

Асимметричное затопление тогда создаёт крен.

29. Комнаты и атмосфера

29.1. Текущая игровая модель

`RoomComponent` хранит normalized oxygen/pressure, temperature, bounds и связи, но не массу газа и физический volume. Поэтому это **игровая диффузионная модель**, не CFD и не thermodynamics solver.

29.2. Парный перенос

Для двух комнат:

```
transfer = (A - B) · rate · doorFlow · dt
```

Transfer ограничивается половиной разности, а изменения сначала собираются в список и применяются затем. Это даёт два плюса:

- A→B и B→A не зависят от порядка обхода;
- закрытая пара сохраняет сумму величины до clamp/источников.

29.3. Leak

Пробоина использует fraction loss:

```
lost = clamp(leak · dt, 0, 0.95)
pressure *= (1 - lost)
oxygen   *= (1 - lost)
```

Это понятно для gameplay, но не учитывает physical volume и нелинейный массовый расход.

29.4. Breathing и generators

Breathing вычитает normalized oxygen на entity, а generator равномерно раздаёт output активным комнатам. Комната в 10 м³ и 1000 м³ реагирует одинаково. Если понадобится физичность, сначала добавить `volumeM3` и хранить moles/mass, затем вычислять fractions.

29.5. Conservation test

Для двух закрытых комнат без source/sink:

```
sumBefore = amountA + amountB
simulate 10 minutes
abs(sumAfter - sumBefore) / sumBefore <= 0.001
```

Если используются normalized fractions, test должен весить комнаты по volume.

30. Энергетика

30.1. Power grid

Каждый ship root имеет свою сеть. Генераторы дают MW, consumers требуют MW, priority определяет порядок. Внутри одного priority используется water-filling: available power делится поровну, полностью насыщенные consumers удаляются, остаток перераспределяется.

30.2. Battery units

Power — MW, energy — MWh:

```
ΔE_MWh = P_MW · dt_seconds / 3600
```

Код `EmergencyBatterySystem` использует именно это преобразование для charge/discharge. Это пример хорошего явного unit contract.

30.3. Brownout

```
powerRatio = suppliedMW / requiredMW
hasPower = powerRatio >= 0.999
brownout = totalSupplied < totalDemand
```

Gameplay subsystem не должен самостоятельно забирать мощность «мимо» grid; он читает `powerRatio`.

30.4. Engine-level развитие

- stable consumer IDs;
- event на переход powered↔brownout, а не каждый кадр;
- граф dependency/isolated grids;
- telemetry history;
- deterministic allocation независимо от unordered order.

31. Damage, cutting и repair

31.1. Damage flow

```
Weapon/raycast
→ DamageEvent
→ DamageSystem
→ health / hull panel / decals / particles / world event
→ breach or destroyed state
```

31.2. Runtime cutting

F6 строит picking ray и вызывает `MeshCuttingTest::CutByRayAsync`. Старое руководство, где указан синхронный `CutByRay`, устарело.

Концептуальные этапы cutting:

1. ray/triangle intersection;
2. выбор области/плоскости разреза;
3. clipping polygons;
4. interpolation новых attributes;
5. triangulation;
6. rebuild CPU/GPU mesh;
7. rebuild physics shape;
8. запись operation ID/history;
9. создание breach/damage representation.

31.3. Интерполяция атрибутов

Для точки на ребре:

```
p(t) = a + t(b-a)
uv(t) = uvA + t(uvB-uvA)
n(t) = normalize(nA + t(nB-nA))
```

Если используется barycentric interpolation внутри triangle:

```
p = uA + vB + wC,  u+v+w=1
```

31.4. Асинхронность

В текущем коде async cutting уже разделён на две фазы. Main thread выбирает цель и снимает CPU-snapshot вершин и индексов. Worker выполняет clipping и triangulation только на этих копиях. Completion возвращается на main thread, проверяет `ActiveComponent` и то, что entity всё ещё ссылается на ту же model, и только после этого меняет D3D buffers, history и Bullet collider.

Async repair добавляет более сильную optimistic validation: worker запоминает `expectedRevision`, а main-thread completion до первой мутации проверяет revision и наличие repair operation у всех целей. Это защищает от применения устаревшей геометрии.

Scene/entity generation token пока не стал общим engine-контрактом. Локальные проверки model pointer и revision закрывают текущие cutting/repair paths, но новая async-система должна либо получить generation token, либо доказать эквивалентную проверку lifetime. Общая модель worker/completion разобрана в главе 35.

31.5. Repair invariants

- одна operation repair не применяется дважды;
- удалённая entity не получает completion;
- GPU buffer меняется только main thread;
- collider перестраивается после mesh;
- save во время repair имеет определённое состояние;
- visual crack, breach simulation и geometry history согласованы.

Часть VI. NPC, данные и инструменты

32. NPC AI pipeline

Фактический порядок:

```
NpcUtilitySystem
→ CrewTaskCoordinator
→ StateSystem
→ CharacterFacingSystem
→ CharacterControllerSystem
→ AnimationController
```

32.1. Utility AI

Idle score примерно `0.1`. Repair breach score сочетает severity и distance; equipment repair — damage, importance и distance. Utility выбирает не только пробоины, как говорит старый NPC guide, но и повреждённое оборудование.

Общая нормализация:

```
score = Σ wi · normalizedCriterion_i
Σ wi = 1
```

Distance utility, например:

```
distanceScore = 1 - clamp(distance / detectionRange, 0, 1)
```

32.2. Hysteresis

Без hysteresis две близкие задачи меняются местами каждый кадр:

```
switch only if newScore > currentScore + threshold
```

Также полезны minimum commitment time и cooldown.

32.3. Manual command

Debug/manual command имеет приоритет. Utility продолжает считать scores для UI, но не перехватывает управление. Это хорошее разделение decision telemetry и authority.

32.4. Чего пока нет как законченной системы

Старая документация описывает fatigue, fear, перенос инструментов и общий World Task Board так, будто они реализованы. На текущем срезе это всё ещё целевая модель, но reservation ремонта уже появился.

`CrewTaskCoordinator` собирает активные `RepairableComponent`, сортирует NPC по `Entity`, исключает hostile/off-duty/manual-command персонажей и закрепляет цель через `CrewTaskClaimComponent`. `CrewMemberComponent` хранит роль, навык, текущую работу и счётчик завершённых работ. Это уже предотвращает ситуацию, когда два дружественных NPC одновременно выбирают одну пробоину или одно оборудование.

Важно точно различать два уровня:

- `NpcUtilitySystem` оценивает индивидуальную полезность ремонта и оставляет диагностический score;
- `CrewTaskCoordinator` выполняет межагентный арбитраж и назначает уникальные цели;
- общего task board для пожаров, лечения, переноски ресурсов и охраны пока нет;
- claim существует только для ремонтируемой цели и пересоздаётся из состояния мира, а не является полноценной долговременной задачей.

Поэтому новый тип коллективной работы следует добавлять либо расширением координатора, либо будущим единым task board. Нельзя создавать второй скрытый `unordered_map<Entity, Entity>` рядом: тогда UI, save/load и другие AI-системы перестанут видеть единственный источник истины.

33. StateSystem и исполнение

Utility отвечает «что делать», StateSystem — «как выполнить».

Типичный repair state:

```
Acquire target
→ build room route
→ build/follow NavMesh path
→ request/open door
→ approach work zone
→ execute timed repair
→ validate target
→ success/fail/cancel
```

State должен иметь:

- enter/exit;
- timeout;
- reason code;
- target validation;
- cancellation cleanup;
- debug-readable progress.

Файл `StateSystem.h` вырос примерно до 1845 строк. Разделение по state handlers уменьшит coupling и compile time.

34. Recast/Detour NavMesh

34.1. Параметры build

```
cellSize = 0.20 m
cellHeight = 0.10 m
agentHeight = 1.8 m
agentRadius = 0.42 m
agentClimb = 0.30 m
maxSlope = 45°
```

Они должны соответствовать capsule controller. Если radius NavMesh меньше физической capsule, найденный путь может не пройти.

34.2. Pipeline

```
input triangles
→ rasterized heightfield
→ compact heightfield
→ erode by agent radius
→ regions
→ contours
→ polygon mesh
→ detail mesh
→ Detour nav data
```

34.3. Query

```
findNearestPoly(start/target)
→ findPath (A*)
→ findStraightPath (funnel-like corners)
```

Лимиты на срезе: около 256 polygon refs и 64 straight points. Длинный путь может быть обрезан.

34.4. Ограничения

- obstacle footprint строится из world AABB, а не oriented geometry;
- doors исключаются из baked geometry;
- нет tile cache/dynamic carving;
- нет явных off-mesh door links;
- fallback может построить только пол без препятствий;
- локальное avoidance выбирает ближайший dynamic AABB, но это не RVO crowd.

34.5. Debug colors

NavMesh/path/obstacles рисуются one-frame debug lines. Цвет fallback-сетки должен отличаться от полноценной, чтобы «путь сквозь мебель» не выглядел загадочным AI-багом.

35. JobSystem и асинхронные конвейеры

> ФАКТ. Эта глава актуализирована после engine-maturity коммита 9913264. Асинхронность в этом движке — не «запустить всё в другом потоке», а явно разделить CPU-подготовку и применение результата к живому engine state.

35.1. Thread affinity: почему нужны две очереди

Worker безопасен только тогда, когда работает с независимыми данными. Сам факт переноса функции в другой поток не делает ECS, Direct3D или Bullet thread-safe.

Работа	Поток	Почему
чтение/parse файла, clipping, frustum test, CPU mesh processing	worker	работа идёт с immutable snapshot или локальным result
публикация CPU-result в completion queue	worker	передаёт владение результатом, но не меняет engine state
ECS structural change	main	component pools и entity lifetime принадлежат main thread
D3D immediate context/GPU buffer creation	main	immediate context и render state имеют одного владельца
Bullet world mutation	physics/main owner thread	world хранит внутренние ссылки на bodies, actions и constraints
ImGui	main	UI живёт в main-frame lifecycle

Отсюда следует главный контракт:

```
main snapshot → worker CPU result → main-thread validation → ECS/D3D/Bullet apply
```

35.2. Владение и lifecycle JobSystem

`ApplicationClass` хранит `JobSystem` как value member и запускает его с четырьмя workers. Число «4» — это конфигурация приложения, а не ограничение класса: `Initialize(workerCount)` принимает любое положительное число.

Важные гарантии lifecycle:

1. Новый или уже остановленный `JobSystem` имеет `stop=true`, поэтому отвергает job, а не кладёт его в очередь без workers.
2. `Initialize(0)` и отрицательное число workers отвергаются.
3. Повторный валидный `Initialize` сначала выполняет `Shutdown`, затем сбрасывает stop/active count и создаёт новые потоки.
4. Если создание одного из потоков бросит исключение, уже созданные workers останавливаются и join-ятся, а `Initialize` возвращает `false`.
5. Деструктор вызывает idempotent `Shutdown`; копирование и перемещение запрещены.

Концептуальная машина состояний:

```
Stopped/rejecting
      | Initialize(N > 0)
      ▼
Running/accepting — Shutdown ──► Draining queued jobs — join ──► Stopped/rejecting
```

35.3. Как worker извлекает job

Worker ждёт condition variable по предикату `stop || !jobs.empty()`. Очередь `jobs` и флаг `stop` защищены одним `queueMutex`. Job извлекается по FIFO, счётчик `activeJobs` увеличивается, после чего мьютекс отпускается. Тяжёлая функция выполняется вне lock, иначе все workers фактически стали бы одним потоком.

```
lock → wait → pop → activeJobs++ → unlock
                        │
                        └→ execute job
                                │
lock ← notify Wait if idle ← activeJobs-- ↵
```

`Shutdown` ставит `stop=true` и будит workers. Worker завершается только при `stop && jobs.empty()`: уже принятые CPU-jobs дренируются, но новые `SubmitJob` отвергает.

35.4. Wait не равен completion fence

В движке две независимые очереди:

- `SubmitJob` кладёт CPU-работу в worker queue;
- `SubmitMainThread` кладёт completion в main-thread queue.

`Wait()` ждёт только `jobs.empty() && activeJobs == 0`. Он не запускает main-thread callbacks. Поэтому «`Wait` вернулся» означает «CPU-фаза закончена», но не «результат уже применён к

сцене». Вызывать `Wait()` из worker job нельзя: текущий job сам учтён в `activeJobs`, поэтому будет ждать собственного завершения.

`ExecuteMainThreadJobs()` под lock меняет shared queue на локальную, затем выполняет callbacks без удержания mutex. Callback, добавленный во время этого прохода, останется до следующего вызова. Это убирает re-entrancy и долгое удержание `mainThreadMutex`.

35.5. Фактические async-конвейеры

Потребитель	Worker-фаза	Main-thread применение	Защита от устаревшего результата
`AssetLoadSystem`	Assimp/file parse и CPU mesh	completion ставит `CpuLoaded`, `Update()` создаёт GPU buffers	tracker и state; completion захватывает raw `this`, поэтому важен shutdown-порядок
async ship import	Assimp читает GLB, готовит nodes, collision proxies и panel batches	ECS/Bullet/GPU создаются фазами с бюджетом 4 ms на кадр	state живёт в `shared_ptr`; `Clear()` отцепляет старый state от новой сцены
`RenderCandidateSystem`	frustum test по snapshot transform/bounds	completion публикует cache visible entities	cache используется только при близких view/projection; иначе fail-open считает entity кандидатом
async cutting	clipping копии CPU mesh	проверка entity/model, GPU apply, history и collider rebuild	active entity + тот же model pointer
async repair	копирование baseline и replay оставшихся cuts	валидация всех целей, GPU apply и collider rebuild	`expectedRevision`, operation ID/state и per-model pending lock

`AssetLoadSystem::RequestMeshLoad` реализован, но в текущем editor spawn path флаг `useAsyncMesh` выключен. Поэтому generic mesh loader — готовый, но не основной runtime path. Активные async-пути сегодня — ship import, render candidates, cutting и repair.

35.6. Порядок внутри кадра и shutdown

В начале `ApplicationClass::Frame` порядок такой:

1. `ExecuteMainThreadJobs()` применяет готовые completions.
2. `AssetLoadSystem::Update(device)` создаёт GPU buffers для состояния `CpuLoaded`.
3. `UpdateAsyncShipImport(..., 4.0f)` тратит ограниченный main-thread budget на текущую фазу импорта.

Порядок shutdown ещё важнее:

```
JobSystem.Shutdown()          // reject new jobs, drain accepted jobs, join workers
ExecuteMainThreadJobs()       // apply every completion produced before join
SceneManager.Clear()          // release ECS/Bullet runtime owners
PhysicsWorld.Shutdown()
AssetLoadSystem.Shutdown()
```

Так raw `this` в asset completion не становится dangling в штатном `ApplicationClass` lifecycle: workers уже join-нуты, completions выполнены, и только затем уничтожаются scene/assets. Вынос `AssetLoadSystem` в другой lifetime потребует новой гарантии.

35.7. Проверяемые контракты

`tests/CoreContractsTests.cpp` теперь доказывает базовый lifecycle `JobSystem`:

- job отвергается до `Initialize`;
- неположительное число workers отвергается;
- запущенные workers выполняют все принятые jobs;
- после `Shutdown` новая работа отвергается;
- повторный `Initialize` снова принимает и выполняет job.

Контракты запускаются для Debug и Release x64 через:

```
.\scripts\verify_engine.ps1 -Configuration All
```

Это уже автоматический test target, но не полное async test suite и не внешний CI.

35.8. Текущие ограничения

1. Исключение из worker job не перехватывается `JobSystem`: обычно это завершит process, а `activeJobs--` не будет выполнен.
2. Исключение из main-thread completion также не изолировано и выйдет в `ApplicationClass::Frame` или `Shutdown`.
3. Нет cancellation token: принятый job при shutdown дренируется, а не отменяется.
4. Нет priority queue, dependency graph, task handle/future и общего канала ошибок.
5. Приложение всегда запускает четыре workers и не учитывает `hardware_concurrency` и нагрузку на render thread.
6. `SubmitMainThread` не проверяет stop-state и не возвращает status; вне штатного application lifecycle completion может остаться без consumer.
7. Нет универсального scene/entity generation token. Cutting и repair имеют локальную валидацию, ship import изолирует state через `shared_ptr`, но новый async-consumer не получает эту защиту автоматически.
8. Static cache `RenderCandidateSystem` не имеет scene generation и не сбрасывается в `SceneManager::Clear`; матричный fail-open снижает риск невидимого объекта, но не заменяет lifecycle token.

Целевая форма completion для новых систем:

```
RequestId + sceneGeneration + entityGeneration/revision + immutable CPU result + error
```

36. Scene save, runtime save и Entity remap

36.1. Текущее scene save

Schema `version=1`, объекты собираются вручную из нескольких pools. `SerializeObject` покрывает много компонентов, затем `LoadScene` создаёт новые Entity и строит map:

```
oldEntityId → newEntityId
```

После создания выполняется `RemapEntityReferences`.

36.2. Подтверждённые пробелы

- loader читает `world`, saver его не пишет;
- `WorldAreaComponent` не сохраняется;
- `SerializeMaterial` — пустой stub;
- sprites не входят в полноценный round trip;
- одна entity может попасть в objects через несколько component pools;
- JSON exception не перехватывается;
- текущая сцена сбрасывается до полной проверки новой;
- прямой Registry reset обходит специальный Bullet teardown;
- version не мигрируется.

36.3. Транзакционная загрузка

```
read bytes
→ parse JSON
→ validate schema/version
→ build temporary scene/registry
→ validate references/resources
→ atomically swap current scene
→ destroy old scene safely
```

36.4. Атомарная запись

```
write save.tmp
→ flush + close
→ rename/replace save.json atomically
```

Повреждение `save.tmp` не уничтожает предыдущий рабочий save.

36.5. Scene и runtime save

Scene asset хранит authoring data. Runtime save дополнительно хранит:

- world tick/time;
- velocities;
- health/damage;
- atmosphere/water/power;
- AI tasks/memory/reservations;
- in-progress operations;
- random seeds.

Смешивать два формата без явной границы опасно.

37. Editor, picking и диагностика

37.1. Редактор

`EditorUI.cpp` — крупнейший hotspot. Он показывает components, создаёт requests, управляет inspectors и debug panels. Хороший принцип уже используется: тяжёлая операция из UI оформляется request и исполняется application/scene layer, а не внутри ImGui draw.

37.2. Gizmo и physics

Editor меняет Transform. Если entity dynamic, physics layer должен распознать teleport, обновить body/AABB, сбросить или сохранить velocities по явной политике. Изменение scale требует rebuild collision shape.

37.3. Диагностический минимум объекта

```
Entity + generation/alive
Name + Active
Transform + units
world bounds
material/shader/textures
Bullet body/shape/type/mass/velocity
room/ship membership
AI state/target/path/reason
save support yes/no
last system that modified critical value
```

37.4. Debug UI не должен быть логикой

UI может вызвать command, но правило симуляции должно жить в system. Иначе headless test и NPC не смогут использовать ту же механику.

Часть VII. Срез качества и план развития

38. Сильные стороны движка

1. Порядок кадра виден в одном координаторе.
2. ECS отделяет данные от большинства систем.
3. Bullet использует fixed step и accumulator clamp.
4. Character controller общий для player/NPC.
5. Async render culling работает по snapshot, не по живому ECS.
6. CPU asset loading отделено от GPU upload.
7. Editor/Play — явные режимы.
8. PBR shader содержит реальную Cook–Torrance математику.
9. Render pipeline уже многопроходный.
10. Entity references remap-ятся после load.
11. Корабельные systems образуют причинные цепочки.
12. Debug overlays существуют для physics/NavMesh/objects.

39. Реестр слабых мест

Приоритет	Слабое место	Симптом	Главная проверка
P0	Release x64 не настроен	C1083/link/subsystem	clean Debug+Release matrix
P0	partial init raw pointers	случайный crash в Shutdown	fail Initialize после каждого шага
P0	scene reset обходит Bullet teardown	crash/dangling после load	100 load/unload + counters
P0	FileSystem path traversal	delete вне assets	`..`/absolute/symlink tests
P0	refraction uninitialized lights/camera	случайный PBR	deterministic render capture
P0	Euler order render/Bullet	collider не совпадает	random compound rotation test
P0	flooding units/double count	слишком медленный/тяжёлый flood	analytical Torricelli/buoyancy tests
P1	transparent depth/order	исчезающее стекло	creation-order invariant image
P1	forces outside fixed tick	FPS-dependent physics	30/60/144 comparison
P1	incomplete serialization	потеря world/material/area	round-trip + corruption test
P1	Entity without generation	stale ID becomes valid	reuse test
P1	normal matrix	неверный свет при scale	inverse-transpose reference
P1	shadow at origin/no skinning	тени исчезают/старая поза	long-distance animated test
P1	current-directory assets	EXE не запускается отдельно	launch from output/temp

Приоритет	Слабое место	Симптом	Главная проверка
P1	jobs without exception/cancel	terminate/hanging Wait	throwing/cancel stress test
P2	monolithic coordinators	дорогие правки/merge conflicts	module dependency budget
P2	duplicate vendored ImGui	drift/size	single dependency source
P2	mixed encoding	C4566/mojibake	UTF-8 compile policy
P2	узкое test coverage / no CI	сложные регрессии всё ещё узнаются вручную	расширять contracts/integration tests + CI gate

39.1. Самые важные пять

Если времени только на пять исправлений:

1. воспроизводимый Release/runtime package;
2. безопасный scene/Bullet teardown;
3. тестовый target;
4. единые rotation/time/fixed-step contracts;
5. refraction/transparency/render-state correctness.

40. Производительность без угадывания

40.1. CPU budgets

Измерять отдельно:

- input/editor;
- AI/NavMesh;
- fixed physics;
- scene systems;
- render preparation/culling;
- draw submission;
- UI;
- main-thread asset upload.

40.2. GPU budgets

- shadow;
- reflection/refraction;
- opaque;
- transparent;
- particles;
- first person;
- postprocess.

40.3. Counters

```
entities/components
Bullet bodies/actions/constraints
render candidates/culled/draw calls/triangles
jobs queued/active/failed
assets queued/ready/failed
GPU buffers/textures/SRV
```

40.4. Hotspot strategy

Сначала profiler, затем изменение. Например, `CULL_NONE` удваивает часть raster workload, но его нельзя просто включить глобально: импортированные winding/inside surfaces должны иметь явный material culling mode.

41. Как диагностировать симптом

Объект не виден

1. Active?
2. Transform finite и scale > 0?
3. model/buffers ready?
4. material/shader/SRV?
5. main/room/frustum culling?
6. transparent queue/depth?
7. camera near/far?

Объект дрожит

1. два владельца Transform?
2. force зависит от render FPS?
3. fixed pose не интерполируется?
4. camera читает pre-physics?
5. animation root motion спорит с controller?
6. collider/pivot offset?

NPC не идёт

1. Utility выбрала target?
2. manual command блокирует?
3. State reason?
4. start/target project on NavMesh?
5. path не обрезан?
6. door links/room route?
7. controller capsule проходит физически?
8. fallback mesh не игнорирует obstacles?

После load crash

1. JSON parse/version;
2. old scene destroyed слишком рано;
3. Bullet action/constraint остался;
4. async completion пришёл из старого generation;
5. Entity reference не remap;
6. material/model pointer уничтожен;
7. static render/nav cache не invalidated.

Значение зависит от FPS

Искать:

- константу «на кадр» без `dt`;
- `lerp(..., 0.3)`;
- силы вне fixed tick;
- random calls разное число раз на frame;
- clamp после интегрирования;
- unordered iteration, влияющий на решение.

42. Практикум: безопасное добавление engine-функции

42.1. Шаблон design note

```
Проблема:
API/данные:
Источник истины:
Владелец:
Поток:
Фаза кадра:
Единицы:
Инварианты:
Сериализация:
Диагностика:
Автотест:
Manual test:
Rollback:
```

42.2. Пример: Heat как engine subsystem

Не начинать с fire particle. Сначала:

1. `HeatComponent` с Kelvin/J/K;
2. `HeatTransferSystem` с pairwise order-independent transfers;
3. fixed/scaled time contract;
4. conservation test;
5. debug heat overlay;
6. serialization;
7. только затем fire gameplay и visual particles.

42.3. Definition of Done системы

- API минимален;
- ownership ясен;
- units названы;
- phase зарегистрирована;
- test падает при намеренной ошибке;
- debug UI объясняет состояние;
- save/load round trip;
- нет D3D/Bullet warnings;
- performance budget известен;
- known limitation записано.

43. Карта чтения кода

Маршрут «запуск и кадр»

```
src/Core/main.cpp
→ src/Core/SystemClass.cpp
→ src/Core/ApplicationClass.cpp::Initialize/Frame/Shutdown
→ src/Scene/SceneManager.cpp::UpdateAll
→ src/Render/Renderer.cpp::RenderFrame
```

Маршрут «сущность»

```
src/ECS/Components/ECSTypes.h
→ ComponentManager.h
→ ECSRegistry.h
→ Components.h
→ SceneManager::SpawnObject/RemoveObject/SerializeObject
```

Маршрут «физика персонажа»

```
CharacterMovementSystem
→ CharacterFacingSystem
→ CharacterControllerSystem
→ PhysicsWorld
→ PhysicsSystem post-step sync
```

Маршрут «NPC»

```
NPCComponents.h
→ NpcUtilitySystem.h
→ NpcCommandSystem.h
→ StateSystem.h
→ NpcNavMeshSystem.cpp
→ CharacterControllerSystem
```

Маршрут «PBR frame»

```
Renderer::RenderFrame
→ SceneManager::RenderAllECS
→ RenderMesh.h
→ MaterialClass
→ PBRShaderClass
→ pbr.vs / pbr.ps
```

Маршрут «корабельная авария»

```
WeaponSystem/DamageSystem
→ HullPanel/Breach
→ FloodingSystem/RoomEnvironmentSystem
→ PhysicsSystem/SubmarineSystem
→ ShipPowerGrid/OxygenGenerator
→ ShipAlert/NpcUtility/RepairSystem
```

Часть VIII. Работа с кодом и практикум по NPC

44. Как изучать собственный движок, а не просто читать файлы

Эта часть превращает справочник в рабочий учебник. Её задача — научить не запоминать тысячи строк, а восстанавливать путь данных и находить правильное место изменения.

44.1. Пять вопросов перед чтением любой системы

Перед тем как разбирать метод, ответь на пять вопросов:

1. **Какие данные он читает?** Например, `NpcUtilitySystem` читает позиции, повреждения, пробоины и настройки мозга.
2. **Какие данные он меняет?** Например, он записывает `repairScore`, `repairTarget`, `state` и `status`.
3. **Кто вызывает метод?** Система сама по себе не работает; важна её позиция в `ApplicationClass::Frame` или `SceneManager::UpdateAll`.
4. **Кто использует результат?** `StateSystem` читает решение utility, а `CharacterControllerSystem` читает уже готовое движение.
5. **Что является источником истины?** Для движения это `CharacterMovement` до физического шага и Bullet-transform после него; для враждебности — `NPCComponent::isHostile`; для назначения ремонта — claim на цели.

Если на эти вопросы нет ответа, изменение пока рано писать. Большинство трудноуловимых багов движка возникает не внутри формулы, а между двумя владельцами одного состояния.

44.2. Чтение по вертикальному пути

Не нужно читать каталог подряд. Выбирай одно наблюдаемое поведение и иди от входа до результата.

Пример для движения NPC:

```
NPCComponent / NpcBrainComponent
→ NpcUtilitySystem или NpcCommandSystem
→ StateSystem
→ CharacterMovement
→ CharacterFacingSystem
→ CharacterControllerSystem
→ Bullet fixed step
→ PhysicsSystem sync/interpolation
→ TransformComponent
→ AnimationController
→ Renderer
```

Пример для ремонта:

```
DamageSystem создаёт повреждение
→ RepairableComponent описывает работу
→ CrewTaskCoordinator закрепляет цель
```

```
→ NpcUtilitySystem считает полезность
→ StateSystem ведёт NPC к WorkZone
→ RepairSystem применяет прогресс
→ WorldEventLog записывает результат
→ HUD/debug объясняет состояние
```

Такой маршрут даёт контекст. Отдельный метод без вызывающей и принимающей стороны почти всегда вводит в заблуждение.

44.3. Метод изучения за 30 минут

Для каждой новой подсистемы используй один и тот же цикл:

1. Найди типы данных через `rg -n "ComponentName|SystemName" .`.
2. Найди место вызова `Update`, `Spawn`, `Destroy` или `Render`.
3. Выпиши producer и consumer каждого изменяемого поля.
4. Включи debug UI и сопоставь поля с наблюдаемым поведением.
5. Измени один параметр через инспектор, не меняя код.
6. Предскажи результат до запуска.
7. Запусти и сравни предсказание с фактом.
8. Верни параметр и запиши вывод в `DEV_NOTES_OFFLINE.md`.

Этот способ полезнее пассивного чтения: ты строишь причинную модель движка и сразу проверяешь её.

44.4. Как отличать реализованное от проектируемого

В документации встречаются три типа утверждений:

- **факт текущего кода** — можно найти тип и место вызова;
- **ограничение** — код работает, но имеет известную границу;
- **целевая архитектура** — идея для следующего этапа, которой ещё нет в runtime.

Пример: `CrewTaskClaimComponent` — реализованный reservation ремонта. Общий task board пожаров, лечения и снабжения — целевая архитектура. Нельзя писать новую систему, исходя из того, что целевая часть уже существует.

44.5. Минимальная design note перед новой механикой

```
Наблюдаемое поведение:
Компоненты данных:
Система, принимающая решение:
Система, исполняющая решение:
Источник истины:
Место в кадре:
Время: real / simulation / fixed:
Что происходит при pause:
Как обрабатывается удаление target:
Что сохраняется в JSON:
Что видно в debug UI:
Минимальная тестовая сцена:
Критерий готовности:
Что намеренно не входит в задачу:
```

Если этот список заполнен, код обычно получается меньше и понятнее.

45. Анатомия персонажа в DX Framework

Игрок, дружественный NPC и враг используют общую физическую основу. Различие задаётся набором компонентов и тем, какая система пишет намерение движения.

45.1. Слои сущности персонажа

Слой	Компоненты	Ответственность
Идентичность	`NameComponent`, `NPCComponent`, `PlayerGameTag`	кто это и какую роль играет
Представление	`MeshRendererComponent`, `AnimationComponent`	модель, материал, скелет и клипы
Положение	`TransformComponent`	отображаемая позиция, поворот и масштаб
Намерение	`CharacterMovement`	куда и с какой скоростью персонаж хочет двигаться
Физика	`CharacterController`, `CharacterPhysicsTuningComponent`	Bullet capsule, земля, ступени, прыжок
NPC-настройки	`NPCMovementComponent`	скорость, бег, дистанции прибытия и дверей
AI runtime	`NpcBrainComponent`	состояние, команда, цели, маршрут и debug
Экипаж	`CrewMemberComponent`	роль, смена, ремонтный навык и assignment

Mesh не определяет игровую принадлежность. Одна и та же модель может быть дружественной или враждебной. Источником истины служат gameplay-компоненты.

45.2. `NPCComponent`

`NPCComponent` уже содержит:

- `name` и `description` для инспектора и взаимодействия;
- `health` и `maxHealth` как базовое здоровье NPC;
- `isHostile` как текущий признак враждебности.

Флаг `isHostile` уже:

- редактируется чекбоксом `Hostile` в `EditorUI`;
- сохраняется как `npc_hostile`;
- восстанавливается при загрузке сцены;
- передаётся в `InteractionTargetInfo::isHostileNpc`;
- меняет цвет панели NPC в `GameHud`;
- исключает NPC из автоматического распределения ремонтных задач.

Поэтому отдельный `EnemyComponent` для первого врага не нужен. Он создал бы два источника истины: `isHostile == true`, но `EnemyComponent` отсутствует, или наоборот. Отдельный компонент станет оправдан только для самостоятельного набора боевых данных, например `HostileAiComponent`, а не как второй флаг принадлежности.

45.3. `NpcBrainComponent`

Brain хранит изменяемое состояние, но не обязан содержать всю будущую логику. Его важные группы полей:

- текущие `NpcState` и `NpcCommand`;
- цель ремонта и комнатный маршрут;
- активная дверь и состояние прохождения портала;
- utility-настройки и диагностические score;
- таймеры перестроения пути и detector застревания;
- состояние wander;
- debug-команды и объясняющие строки.

Поле `status` предназначено для ответа «что сейчас происходит», а `blockedReason` — «почему система не может продолжить». Они не должны быть условием gameplay-логики.

45.4. `CharacterMovement` как граница между AI и физикой

AI не должен напрямую двигать `TransformComponent` или вызывать Bullet `warp` для обычной ходьбы. Он записывает намерение:

```
walkDirection
currentSpeed
isMoving
wantsRun / isRunning
wantJump
locomotionState
```

После этого общие системы выполняют намерение. Благодаря такой границе:

- игрок и NPC используют одну collision-модель;
- анимация получает одинаковые состояния locomotion;
- враг не проходит сквозь препятствия особым способом;
- AI можно тестировать отдельно от Bullet.

45.5. Создание runtime-контроллера

Добавление `CharacterController` в ECS ещё не создаёт Bullet-объекты. `CharacterSpawnSystem::Spawn`:

1. проверяет `CharacterController` и `TransformComponent`;
2. уничтожает старый runtime controller при повторном spawn;
3. создаёт capsule shape;
4. создаёт ghost object и связывает его с Entity через `userIndex`;
5. создаёт `btKinematicCharacterController`;
6. применяет gravity, slope, jump и penetration settings;
7. регистрирует collision object и action в Bullet world.

Парный `Destroy` обязан удалить action и collision object из мира до освобождения памяти. Простое удаление ECS-компонента без teardown опасно.

45.6. Что делает `CharacterControllerSystem::Update`

Метод обслуживает все сущности с `CharacterController`:

- останавливает и временно снимает Bullet-объекты неактивной сущности;
- лениво добавляет безопасный `CharacterPhysicsTuningComponent` старым сценам;
- вычисляет опору геометрическим sweep, а не доверяет только `onGround()`;
- применяет auto-step;
- объединяет locomotion и внешнюю скорость декомпрессии;
- запускает прыжок;
- переводит неподвижного стоящего персонажа в стабильный physics idle;
- очищает runtime-map для удалённых сущностей.

Публичные диагностические методы `IsAutomaticStepActive`, `HasGroundSupport`, `IsIdlePhysicsLocked` позволяют UI показать внутреннее состояние без дублирования расчётов. `ForceReconfigure` сбрасывает runtime cache после изменения настроек в инспекторе.

45.7. Поворот и анимация

`CharacterFacingSystem` читает горизонтальное направление движения и плавно меняет visual rotation. Он не принимает решение, куда идти.

`AnimationController` читает locomotion-результат и выбирает Idle, Walk, Run или другой доступный клип. AI не должен напрямую запускать Walk каждый кадр: это связывает решение с конкретным ассетом и ломает fallback для моделей с другим набором клипов.

46. Дружественный NPC в текущей архитектуре

46.1. Состав дружественного NPC

Минимальная дружественная сущность содержит:

```
ActiveComponent
TransformComponent
MeshRendererComponent
AnimationComponent
NPCComponent { isHostile = false }
NpcBrainComponent
NPCMovementComponent
CharacterMovement
CharacterController
CharacterPhysicsTuningComponent
```

`CrewMemberComponent` может быть добавлен вручную или будет создан координатором с defaults. Для предсказуемой сцены лучше авторить его явно и сохранять роль, `onDuty` и `repairSkill`.

46.2. `NpcCommandSystem`

Это единая точка выдачи ручного приказа. Команды не исполняют движение сами: они валидируют/записывают `NpcCommand`, увеличивают revision и сбрасывают необходимые runtime-поля.

Назначение методов:

Метод	Смысл	Время жизни команды
`Stop`	запретить автоматике перехватить NPC	пока не будет Resume/другая команда
`Wander`	постоянно выбирать точки блуждания	постоянная
`MoveToPosition`	прийти в мировую точку	снимается по прибытии
`MoveToRoom`	прийти внутрь комнаты	снимается после безопасного прохода портала
`FollowEntity`	следовать за движущейся Entity	постоянная
`RepairEntity`	прийти и починить конкретную цель	снимается после завершения/невалидности
`ResumeAutomatic`	очистить ручной приоритет	немедленно возвращает authority автоматике
`ForceRepath`	инвалидировать маршрут	не меняет смысл текущего приказа

Главный инвариант: `command.type != None` означает, что automatic utility не имеет права менять задачу NPC. Debug UI не должен обходить этот контракт прямой записью `brain.state`.

46.3. `NpcUtilitySystem`

Utility проходит по NPC с brain, movement и transform, оценивает ремонтируемые пробоины и оборудование. При оценке учитываются валидность, расстояние, тяжесть повреждения и чужой claim.

Система всегда может вычислять score для диагностики, но при активной ручной команде не меняет решение. Это позволяет видеть, что автоматика выбрала бы, не отбирая управление у разработчика.

46.4. `CrewTaskCoordinator`

Координатор решает межагентную проблему: несколько NPC не должны чинить одну цель.

Последовательность `Update`:

1. очистить невалидные claims;
2. собрать и отсортировать ремонтируемые цели;
3. собрать активных NPC и отсортировать по Entity для воспроизводимости;
4. исключить hostile, off-duty и NPC с ручной командой;
5. сохранить действующее валидное назначение;
6. оценить свободные цели;
7. добавить `CrewTaskClaimComponent` цели;
8. записать assignment в `CrewMemberComponent` и brain;
9. обновить debug snapshot.

`IsRepairTargetValid` — общий контракт валидности цели. `IsClaimedByOther` используется utility, чтобы не выбирать чужую работу. `GetSnapshot` отдаёт UI количество доступного экипажа, занятых работников и открытых задач.

Роль влияет на score: инженер получает преимущество в ремонте, Operations — дополнительную близость к генераторам и кислородному оборудованию, Security/Medic имеют меньшую склонность к обычному ремонту. `repairSkill` влияет не только на выбор, но и на скорость исполнения ремонта.

46.5. `StateSystem`

`StateSystem` — исполнитель. Он не должен решать стратегическую полезность задачи.

Верхний `Update` проверяет обязательные компоненты и направляет NPC либо в manual-command handler, либо в automatic state. Для movement-задач используется общий `NavigateTo`:

```
определить текущую и целевую комнаты
→ построить маршрут по дверям
→ выбрать approach/exit точки портала
→ запросить открытие двери
→ пройти портал полностью
→ запросить локальную NavMesh waypoint
→ записать CharacterMovement
→ следить за фактическим прогрессом
```

`FollowEntity` уже является полезным прототипом преследования: target transform читается каждый кадр, target room обновляется, а NPC держит дистанцию 1.5 м. Но это ручная бесконечная команда, а не hostile perception/combat AI.

`UpdateRepairTask` унифицирует пробоину и оборудование через `RepairableComponent`. Для оборудования он ищет `WorkZoneComponent`, выбирает достижимую рабочую точку, подходит, учитывает `repairSkill` и вызывает общий ремонтный путь.

46.6. Двери и NavMesh

Комнатный маршрут и локальный NavMesh решают разные задачи:

- граф комнат говорит, через какие двери идти;
- NavMesh говорит, как пройти внутри комнаты вокруг статических препятствий;
- local dynamic avoidance даёт короткий объезд динамического AABB;
- Bullet controller окончательно решает физическую проходимость.

Успешный путь NavMesh не гарантирует, что capsule пройдёт: размеры агента и физического контроллера должны совпадать. При любом застревании сначала смотри `status`, `blockedReason`, door snapshot, current room, waypoint и ground support.

46.7. Что значит «почти идеальный дружественный NPC»

Для текущего масштаба проекта это не NPC с сотней реплик. Это персонаж, который:

- стабильно стоит, ходит, бегает и проходит допустимые пороги;
- не дрожит и не проваливается;
- проходит двери и восстанавливается после временной блокировки;
- выполняет manual-команды;
- самостоятельно получает уникальную ремонтную работу;
- корректно освобождает claim при отмене, off-duty, уничтожении цели или смене команды;
- имеет понятную роль и параметры в инспекторе;
- сохраняет authoring-данные и безопасно пересоздаёт runtime-данные;
- в каждый момент объясняет состояние через UI;
- не зависит от присутствия игрока или камеры.

47. Проектирование врага на существующем NPC pipeline

47.1. Что уже готово

Для врага уже можно переиспользовать:

- `NPCComponent::isHostile` как принадлежность;
- `health/maxHealth` как базовое здоровье;
- `NpcBrainComponent` как общий navigation/debug runtime;
- `NPCMovementComponent` как параметры движения;
- `CharacterMovement`, controller, facing и animation;
- `FollowEntity`/`NavigateTo` как доказанный путь преследования;
- `PlayerGameTag` как способ найти игрока;
- `DamageSystem` как общий путь применения урона;
- scene save для базовых NPC-полей.

Врагу не нужна отдельная физика или особая телепортация. Новая часть — принятие боевого решения и безопасная атака.

47.2. Почему одного `isHostile` недостаточно

`isHostile` отвечает только «враг ли это». Боевые runtime-данные имеют другой смысл и другое время жизни. Их лучше хранить в отдельном компоненте, например концептуальном `HostileAiComponent`:

```
// Учебный эскиз контракта, не готовая вставка.
struct HostileAiComponent
{
    Entity target = INVALID_ENTITY;
    float detectionRange = 12.0f;
    float loseTargetRange = 18.0f;
    float attackRange = 1.6f;
    float attackCooldownSeconds = 1.0f;
    float cooldownRemaining = 0.0f;
    // Состояние perception/combat добавь осознанно.
};
```

Этот компонент не дублирует `NPCComponent::isHostile`. Маркер отвечает за отношение, hostile AI — за настройки и runtime боя.

47.3. Минимальный автомат состояний

Первый враг должен иметь только наблюдаемые состояния:

```
Idle
→ обнаружил игрока
Chase
→ вошёл в attack range
Attack
→ игрок отошёл
Chase
→ игрок потерян или невалиден
Idle
```

Дополнительное `Blocked` можно переиспользовать из общего navigation runtime либо показать как причину, не превращая первую версию в огромный state machine.

47.4. Perception первой версии

Первый шаг — дистанция и валидность Entity. Затем добавляется line of sight через raycast.

Порядок проверки:

1. существует ли игрок;
2. активен ли он и имеет ли Transform/health;
3. находится ли он в detection/lose range;
4. не закрывает ли raycast стену или дверь;
5. нужно ли удержать уже найденную цель по hysteresis.

Два радиуса обязательны:

```
detectionRange < loseTargetRange
```

Иначе враг будет переключать Chase/Idle каждый кадр на границе одного радиуса.

47.5. Преследование

Не копируй навигацию из `StateSystem`. Вынеси или переиспользуй существующий путь так, чтобы hostile decision только сообщал:

```
target entity
desired stop distance
desired movement state
```

Для первого прототипа допустимо доказать маршрут существующей ручной командой `FollowEntity`. Но автоматическая hostile-система не должна каждый кадр заново выдавать command и увеличивать revision: это постоянно инвалидирует маршрут. Ей нужен стабильный combat state или отдельный вызов общего navigation helper.

47.6. Атака

Первая атака должна быть простой melee-атакой с cooldown. Условия применения урона проверяются в момент удара, а не только при входе в Attack:

- target всё ещё валиден;
- target всё ещё в attack range;
- при необходимости line of sight не потерян;
- cooldown завершён;
- урон проходит через общий `DamageSystem`/damage event, а не прямое изменение случайного поля HP.

Cooldown использует `simulationDeltaTime`. На паузе он не уменьшается. Один удар не должен зависеть от FPS и не должен наноситься каждый кадр анимации.

47.7. Порядок hostile-системы в кадре

Боевой decision producer должен работать до facing/controller/animation. Возможный контракт:

```
NpcUtilitySystem (friendly repair score)
→ CrewTaskCoordinator (friendly assignment)
→ Hostile decision/perception
→ StateSystem или общий navigation executor
→ CharacterFacingSystem
→ CharacterControllerSystem
→ AnimationController
→ Damage resolution / particles
```

Главное правило: в одном кадре только один слой имеет authority над `CharacterMovement` конкретного NPC. Для hostile NPC дружественный repair pipeline должен быть выключен — это уже частично обеспечивает `CrewTaskCoordinator`, но `NpcUtilitySystem` тоже нужно проверить на ранний выход для `isHostile`.

47.8. Смерть и потеря цели

Вражеский AI обязан безопасно переживать:

- удаление игрока;
- выход из Play Mode;
- загрузку новой сцены;
- деактивацию врага;
- смерть врага;
- смерть target;
- уничтожение двери/изменение маршрута.

Нельзя хранить raw pointer на компонент цели между кадрами. Храни `Entity`, а компонент получай заново после проверки `IsAlive` и наличия нужного типа.

47.9. Debug-контракт врага

Для выбранного врага показывай:

```
Hostile: yes/no
Combat state
Target Entity
Distance to target
Detection / lose / attack ranges
Line of sight
Cooldown remaining
Current room / target room
Nav status / waypoint
Movement intent
Blocked reason
Last damage result
```

Если враг «не работает», эта панель должна за минуту ответить: он не видит, не решил, не построил путь, физически застрял или не прошёл условие удара.

48. Практическое задание Первый полноценный дружественный NPC

Цель задания — довести одного члена экипажа до состояния, при котором его можно оставить в сцене без постоянного ручного контроля. Код выполняет разработчик; учебник задаёт архитектурные границы и проверяемый результат.

48.1. Сценарий

Создай NPC `Crew_Engineer_01`. Он стоит в жилой комнате, получает роль Engineer, самостоятельно выбирает одну из двух ремонтных целей, проходит в техническую комнату через дверь, чинит цель, освобождает assignment и возвращается в понятное Idle-состояние. Ручная команда временно перехватывает его и после Resume возвращает автоматике.

48.2. Этап 1. Authoring и сохранение

Настрой сущность через редактор:

- читаемые name/description;
- `isHostile = false`;
- корректный набор character/NPC-компонентов;
- роль Engineer, `onDuty = true`, осмысленный `repairSkill`;
- скорости walk/run и arrival distances;
- capsule и physics tuning под размеры модели;
- AnimationComponent с Idle/Walk/Run;
- ActiveComponent.

Сохрани сцену, перезапусти и проверь, что authoring-поля восстановились. Runtime pointer Bullet, текущий путь и временный claim не должны сериализоваться как адреса.

48.3. Этап 2. Физическая стабильность

Проверки:

1. 60 секунд Idle на ровном полу.
2. Проход порогов 0.05, 0.10, 0.20, 0.30 и 0.36 м.
3. Движение по склону до max slope и отказ на слишком крутом.
4. Остановка у цели без колебания вперёд-назад.
5. Исчезновение пола корректно снимает grounded/idle lock.
6. 30, 60 и 144 FPS не меняют заметно скорость и результат.

Запиши capsule bottom, ground Y, grounded, idle lock, auto-step и горизонтальную скорость.

48.4. Этап 3. Команды

Проверь все команды инспектора:

- Stop удерживает NPC;
- Wander выбирает достижимые точки;
- MoveToPosition завершается;
- MoveToRoom проходит портал до безопасной точки;
- Follow player держит дистанцию и не толкает capsule;
- Repair entity работает для пробоины и оборудования;
- Resume auto действительно снимает manual authority;
- Force repath не стирает смысл команды.

Для каждой ошибки сначала исправляй объясняющее состояние, затем поведение. Не оставляй silent failure.

48.5. Этап 4. Автономный ремонт и reservation

Размести две цели: пробоину и повреждённый генератор. Добавь второго простого crew NPC только для проверки arbitration.

Критерии:

- каждый NPC получает максимум одну работу;
- одна цель принадлежит максимум одному NPC;
- Engineer имеет предсказуемое преимущество при близких условиях;
- ручная команда освобождает прежний claim;
- off-duty освобождает claim;
- завершение ремонта увеличивает completed count один раз;
- исчезновение цели не оставляет вечный claim;
- hostile NPC не получает crew assignment.

48.6. Этап 5. Двери, препятствия и восстановление

Проверки:

- закрытая рабочая дверь открывается;
- NPC ждёт створки, а не толкает её;
- после прохода collider двери возвращается;
- временное препятствие вызывает repath/avoidance;
- обесточенная или непроходимая дверь даёт понятный `Blocked`;
- после исправления причины Force repath восстанавливает задачу.

48.7. Definition of Done дружественного NPC

- Debug и Release собираются.
- Нет дрожания в Idle и постоянного толкания двери.
- Все ручные команды имеют предсказуемое завершение.
- Автоматический ремонт работает без камеры и игрока рядом.
- Два NPC не чинят одну цель.
- Save/load сохраняет authoring и не сохраняет опасные runtime pointers.
- В debug UI видно состояние, цель, assignment, путь и причину блокировки.
- Есть отдельная regression-сцена и записанные шаги воспроизведения.
- Известные ограничения перечислены, а не спрятаны.

49. Практическое задание Первый преследующий и атакующий враг

Цель — создать `Enemy_Hunter_01`, который автоматически обнаруживает игрока, преследует его через комнаты и двери, останавливается на дистанции атаки, наносит дискретный урон и безопасно теряет цель.

49.1. Ограничение первой версии

В задачу входят только:

- один игрок;
- один melee-враг;
- Idle, Chase и Attack;
- distance perception и затем line of sight;
- существующая navigation/physics/animation pipeline;
- cooldown и урон;
- debug UI и сохранение настроек.

Не входят patrol graph, ranged weapon, cover, squad tactics, loot, ragdoll, hearing, stealth meter и сложная анимационная синхронизация. Они добавляются после стабильного vertical slice.

49.2. Этап 1. Сущность и данные

Создай NPC через существующий `NPC AI Components`, поставь `Hostile`, настрой mesh, animation, controller и movement. Проверь, что после save/load `npc_hostile` остаётся true и HUD показывает hostile-оформление.

Спроектируй отдельные combat-поля. Перед кодом письменно реши:

- где хранится combat state;
- кто владеет target Entity;
- чем отличаются detection и lose range;
- где хранится attack range, damage и cooldown;
- какие поля authoring, а какие runtime;
- какие из них сохраняются;
- кто имеет authority над `CharacterMovement` hostile NPC.

49.3. Этап 2. Доказать преследование без нового AI

Через debug нажми `Follow player`. Проверь путь на ровном полу, через дверь и между комнатами. Это отделяет проблемы существующей навигации от ошибок будущего combat decision.

Если ручное Follow уже не работает, hostile-код пока не писать. Сначала исправляется общий pipeline.

49.4. Этап 3. Автомат Idle и Chase

Добавь систему принятия hostile-решения. Она обрабатывает только активных NPC с `isHostile == true` и нужными combat-данными.

Требования:

- игрок ищется через `PlayerGameTag`;
- target хранится как `Entity`, не pointer;
- detection использует горизонтальную или трёхмерную дистанцию по явно выбранному правилу;
- захваченная цель теряется только за `loseTargetRange`;
- состояние не переустанавливается каждый кадр без причины;
- изменение target/состояния инвалидирует путь один раз;
- при отсутствии игрока движение полностью останавливается;
- friendly utility не перезаписывает hostile movement.

49.5. Этап 4. Line of sight

Добавь raycast от осмысленной точки врага к осмысленной точке игрока. Ray не должен начинаться в полу или центре capsule, если там мешает собственный collider. Игнорируй entity врага и корректно интерпретируй первое попадание.

Проверки:

- видит игрока в пустой комнате;
- не видит через стену;
- закрытая дверь перекрывает видимость;
- открытая дверь разрешает видимость;
- краткая потеря LOS не вызывает дрожание состояния — при необходимости добавь grace timer;
- debug line показывает реальный луч.

49.6. Этап 5. Attack

Когда дистанция меньше attack range, враг останавливает locomotion и входит в Attack. Удар происходит только по cooldown и повторно проверяет target/range/LOS.

Критерии:

- один удар уменьшает здоровье ровно один раз;
- частота удара одинакова при 30/60/144 FPS;
- pause замораживает cooldown и атаку;
- враг не наносит урон через стену;
- игрок, вышедший из attack range, переводит врага обратно в Chase;
- удалённый/мёртвый игрок очищает target;
- damage идёт через общий механизм, а не обходной прямой доступ к UI-полю;
- friendly NPC не становится жертвой, пока это явно не предусмотрено.

49.7. Этап 6. Анимация и обратная связь

Idle/Walk/Run остаются под управлением общего animation controller. Для первой атаки допустим отдельный trigger/state, но урон нельзя случайно привязать к render frame.

Минимальная обратная связь:

- имя и красный hostile accent;
- combat state в debug;
- attack cooldown;
- короткое событие/лог успешного удара;
- видимое изменение HP игрока;
- понятная причина потери цели.

49.8. Матрица тестов врага

Ситуация	Ожидаемый результат

Ситуация	Ожидаемый результат
Игрок вне detection	Idle, target invalid
Игрок входит в detection	target захвачен, Chase
Игрок между detection и lose после захвата	Chase продолжается
Игрок за стеной	нет захвата или срабатывает grace по выбранному правилу
Игрок в другой связанной комнате	путь идёт через дверь
Игрок ближе attack range	движение остановлено, Attack
Игрок отступил	возврат в Chase
Игрок удалён	target очищен, Idle
Враг деактивирован	Bullet/action и AI не продолжают работу
Игра на pause	движение, cooldown и урон заморожены
Save/load	authoring сохранён, runtime target безопасно сброшен/проверен

49.9. Definition of Done врага

- Враг определяется существующим `isHostile`.
- Combat-данные не дублируют brain/movement без причины.
- Преследование переиспользует общий navigation pipeline.
- Есть hysteresis обнаружения и cooldown атаки.
- Урон применяется дискретно и FPS-независимо.
- Friendly crew pipeline не управляет врагом.
- Невалидная target Entity никогда не разыменовывается.
- Debug UI объясняет perception, decision, navigation и attack.
- Есть тестовая сцена с комнатой, стеной и дверью.
- Все сценарии матрицы проверены и записаны.

50. План одного рабочего дня и порядок сдачи

Если на практику есть один длинный вечер, не пытайся закончить обоих персонажей сразу.

Блок 1. 30 минут — baseline

- собрать Debug;
- сохранить копию тестовой сцены;
- проверить текущего NPC через Follow player и Repair entity;
- записать исходные баги отдельно от новых.

Блок 2. 90 минут — дружественный NPC

- полный authoring;
- роль Engineer;
- команды;
- одна автономная ремонтная задача;
- debug snapshot;
- save/load.

Если на этом этапе есть баг движения или двери, остановись и локализуй его. Враг использует тот же фундамент.

Блок 3. 60 минут — данные врага и Idle/Chase

- `isHostile`;
- отдельный combat runtime;
- target игрока;
- detection/lose hysteresis;
- автоматическое преследование;
- отсутствие конфликтов с crew utility.

Блок 4. 60 минут — Attack

- attack range;
- остановка;
- cooldown;
- общий damage path;
- pause/FPS проверки;
- debug-поля.

Блок 5. 30 минут — фиксация результата

Запиши:

```
Дата:
Сцена:
Изменённые файлы:
Что умеет Crew_Engineer_01:
Что умеет Enemy_Hunter_01:
Какие состояния проверены:
Какие значения ranges/cooldown использованы:
Что происходит при pause:
Что происходит при потере/удалении target:
Известные проблемы:
Следующий маленький шаг:
```

Для code review подготовь короткое видео или последовательность скриншотов и список шагов воспроизведения. Хороший результат — не «вроде бегает», а повторяемый сценарий, в котором каждое состояние можно объяснить данными.

Приложение A. Горячие клавиши

Клавиша	Действие
`F5`	Editor ↔ Play
`F1`	object debug window
`F2`	курсор ImGui в Play / возврат camera capture
`F6`	async mesh cut по picking ray
`Space`	pause/resume через TimeManager
`K`	single step в pause/step mode
`Esc`	выход

Конкретные mouse/weapon/interaction bindings проверять в `InputManager`, `GamePlayController`, `WeaponSystem` и `InteractionSystem`: они меняются чаще архитектуры.

Приложение B. Формулы на одной странице

```
length(v) = sqrt(dot(v,v))
normalize(v) = v / length(v)
W = S·Rx·Ry·Rz·T                    [текущий Transform]
p_clip = p_local·W·V·P              [row-vector]
N' = normalize(N·transpose(inverse(W3x3)))
t_exponential = 1 - exp(-lambda·dt)
fixed alpha = accumulator / h
P_water = P0 + rho·g·h
v_breach = sqrt(2·g·h)
Q = Cd·A·v
F_buoyancy = rho·g·Vdisplaced
torque = r × F
DeltaE_MWh = P_MW·dt_seconds/3600
Fresnel = F0 + (1-F0)(1-cosTheta)^5
specular = D·G·F / (4(N·V)(N·L))
particle uniform radius = R·cbrt(u)
```

Приложение C. Словарь

Термин	Смысл
ECS	Entity IDs + data components + systems
source of truth	единственное авторитетное состояние
RAII	ресурс освобождается временем жизни объекта
fixed step	симуляция одинаковыми шагами времени

Термин	Смысл
interpolation alpha	доля между двумя physics poses
frustum	объём видимости камеры
BRDF	модель отражения света поверхностью
PBR	physically based shading assumptions
IBL	освещение от environment map
PCF	усреднение shadow comparisons
NDC	координаты после perspective divide
skinning	деформация vertices matrices костей
NavMesh	polygon graph проходимой поверхности
remap	old Entity ID → new Entity ID после load
transaction	либо операция завершена целиком, либо старое состояние сохранено
soak test	длительный автоматический прогон на leaks/drift

Приложение D. Что открыть через год

1. Прочитать главы 1, 5, 7 и 12.
2. Собрать Debug/Release по главе 6 и записать новый baseline.
3. Открыть `ApplicationClass::Frame` и сравнить timeline с главой 7.
4. Запустить regression scene и debug overlays.
5. Проверить реестр слабых мест главы 39: что уже исправлено.
6. Только после этого добавлять новую систему.

> Главный вывод: DX Framework уже содержит устройство настоящего движка — composition root, ECS, fixed physics, multi-pass renderer, asset pipeline, editor и связанные simulation systems. Чтобы он стал «конфеткой», следующая ступень — не максимальное число функций, а единые контракты, безопасное владение, воспроизводимая сборка и тесты, которые доказывают математику.